TERRAFORM TASK - 01

1.Install Terraform on your PC

Install Terraform on Your PC — Windows

1. Download Terraform

Open the official HashiCorp Terraform installation page:

Terraform Installation

2. Extract the ZIP file

After downloading, extract it.

You will get:

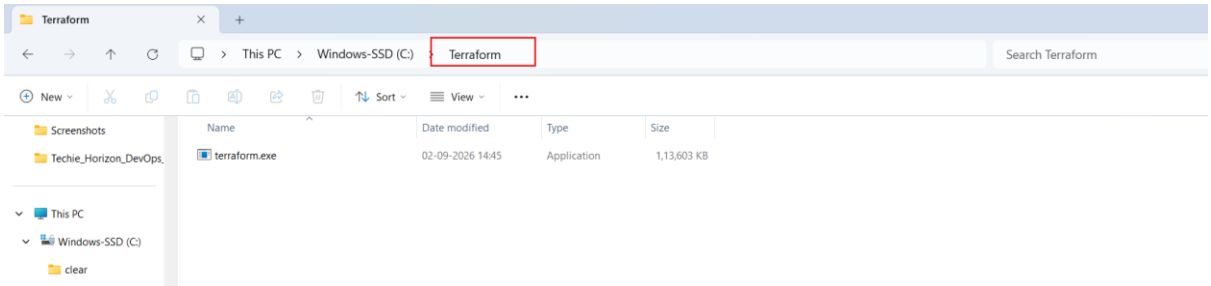
terraform.exe

Create a folder:

C:\Terraform

Move terraform.exe into:

C:\Terraform\terraform.exe



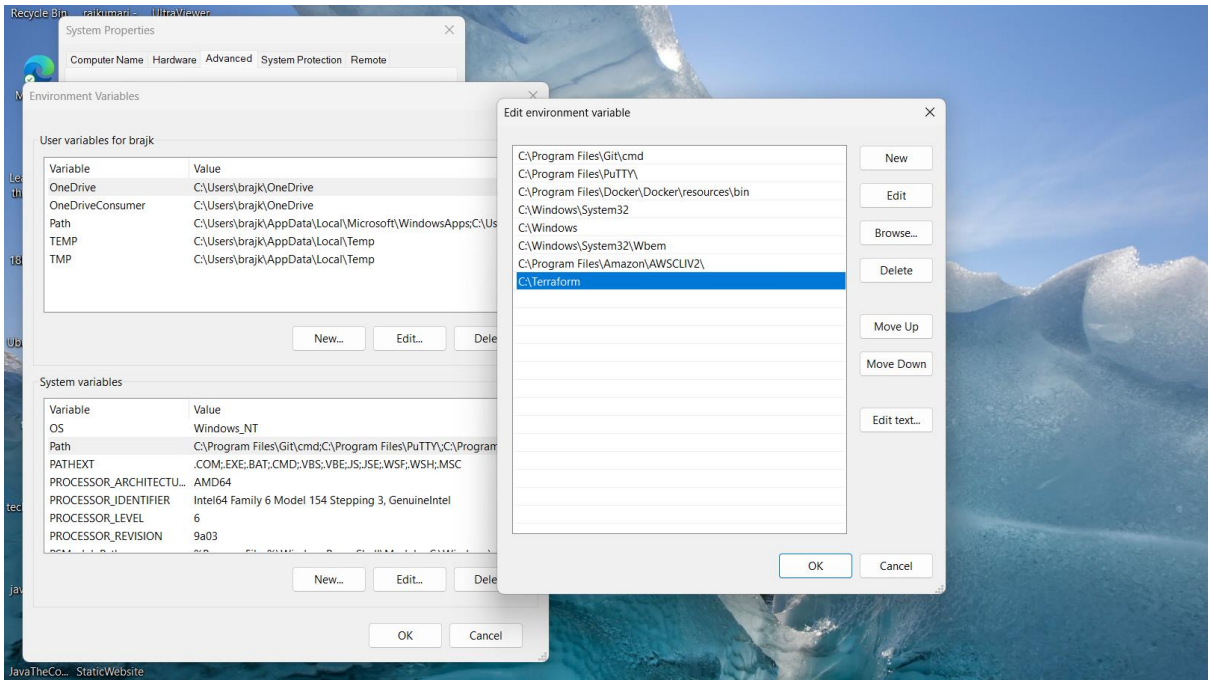
3. Add Terraform to PATH

1. Press **Windows + S**
2. Search for **Environment Variables**
3. Click **Edit the system environment variables**
4. Click **Environment Variables**
5. Under **System variables**, select **Path**

6. Click **Edit**
7. Click **New**
8. Add:

C:\Terraform

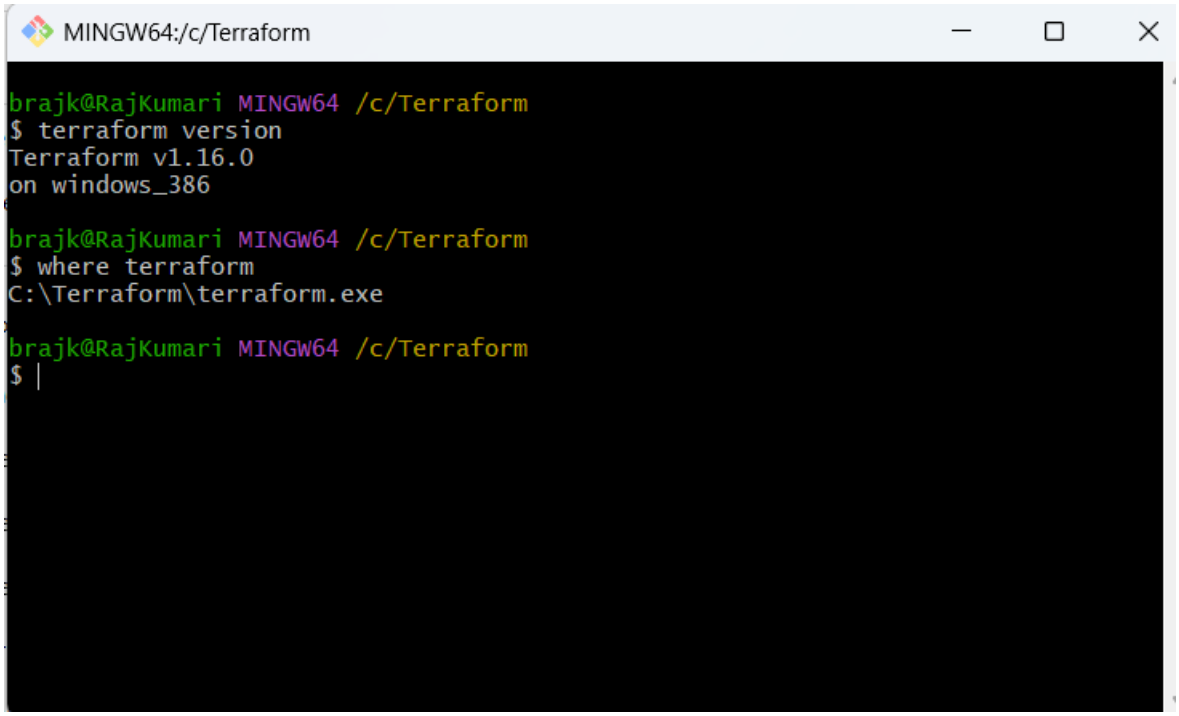
9. Click **OK** on all windows.



4. Verify Terraform installation

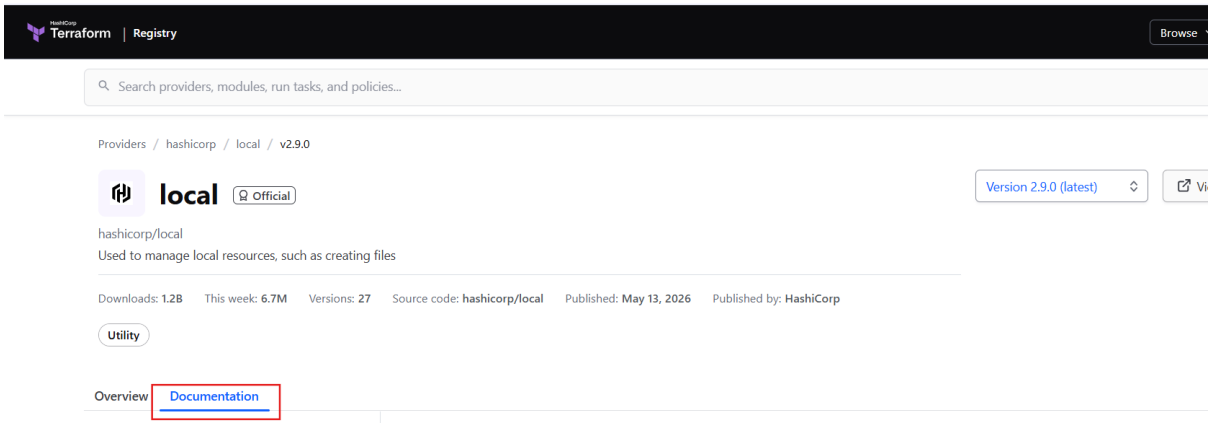
Open a **new Command Prompt** or PowerShell window or git bash

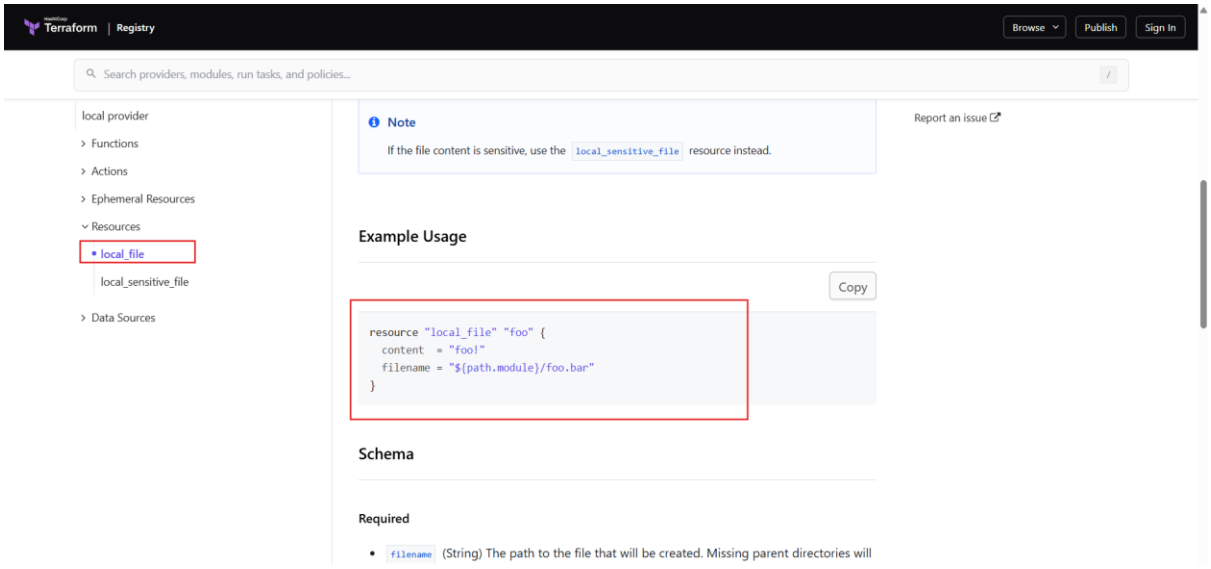
Run: terraform version and where terraform

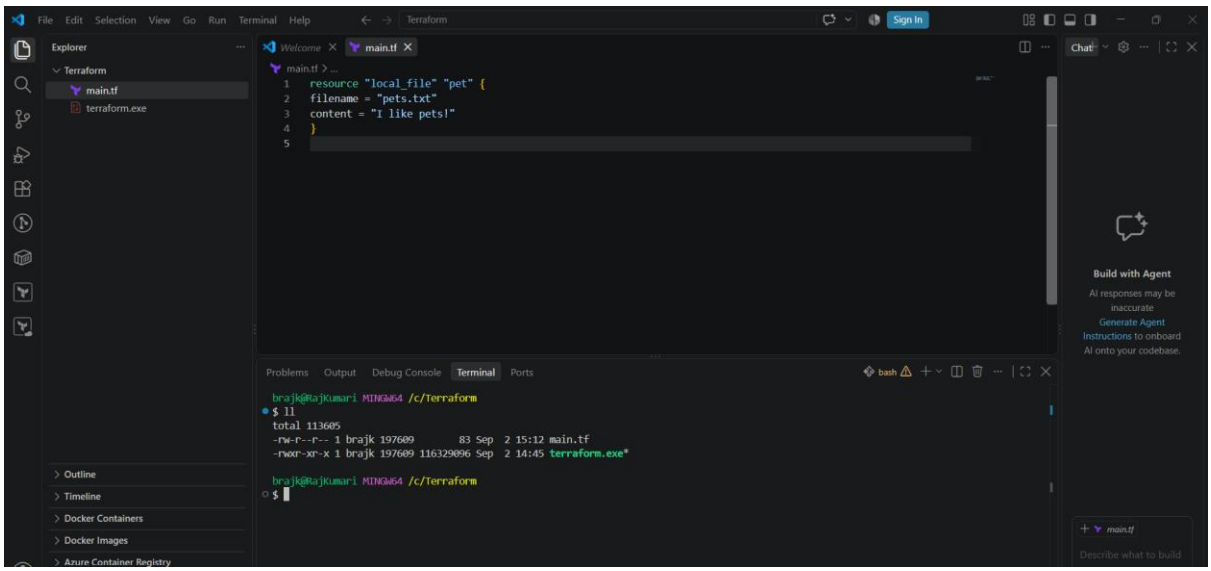


2. Execute all the templates shown in video.

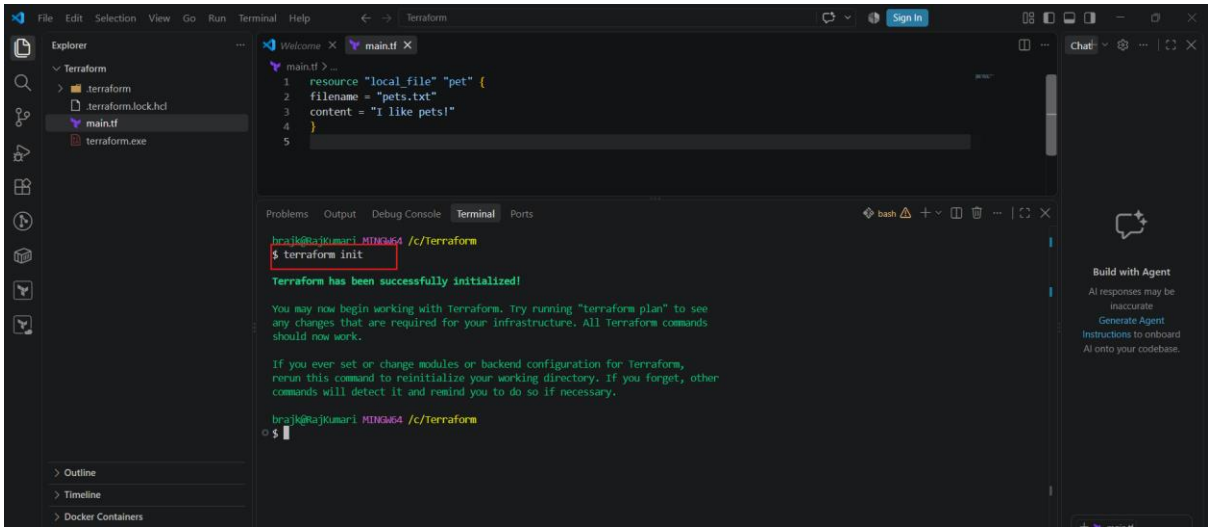
First Template is main.tf using local provider



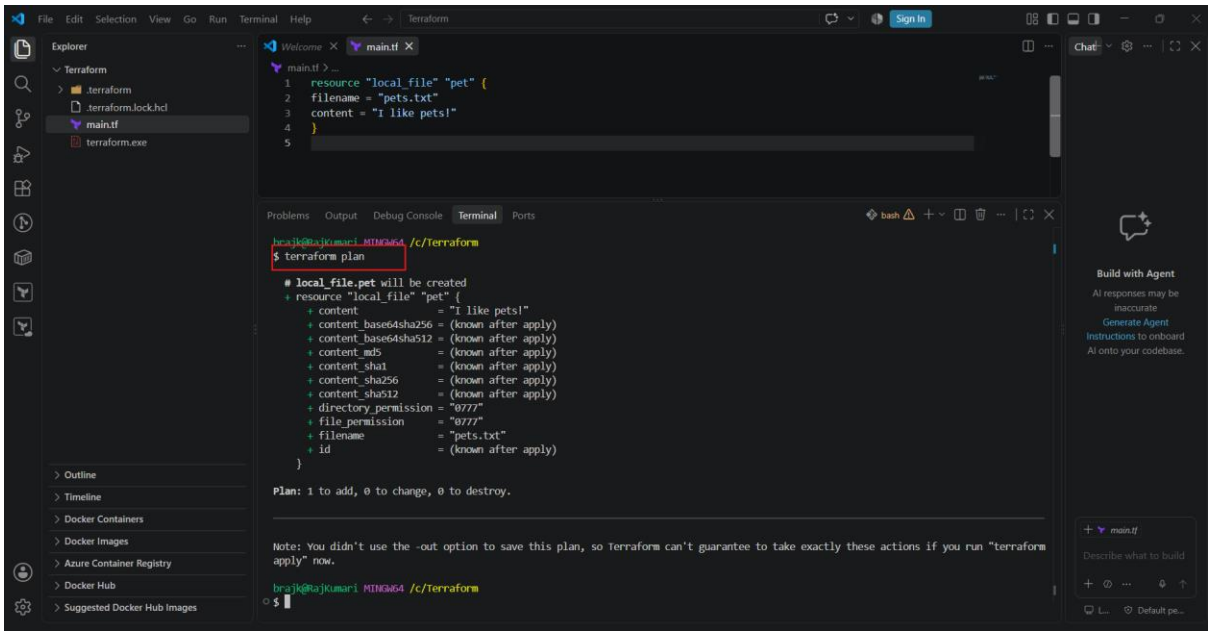




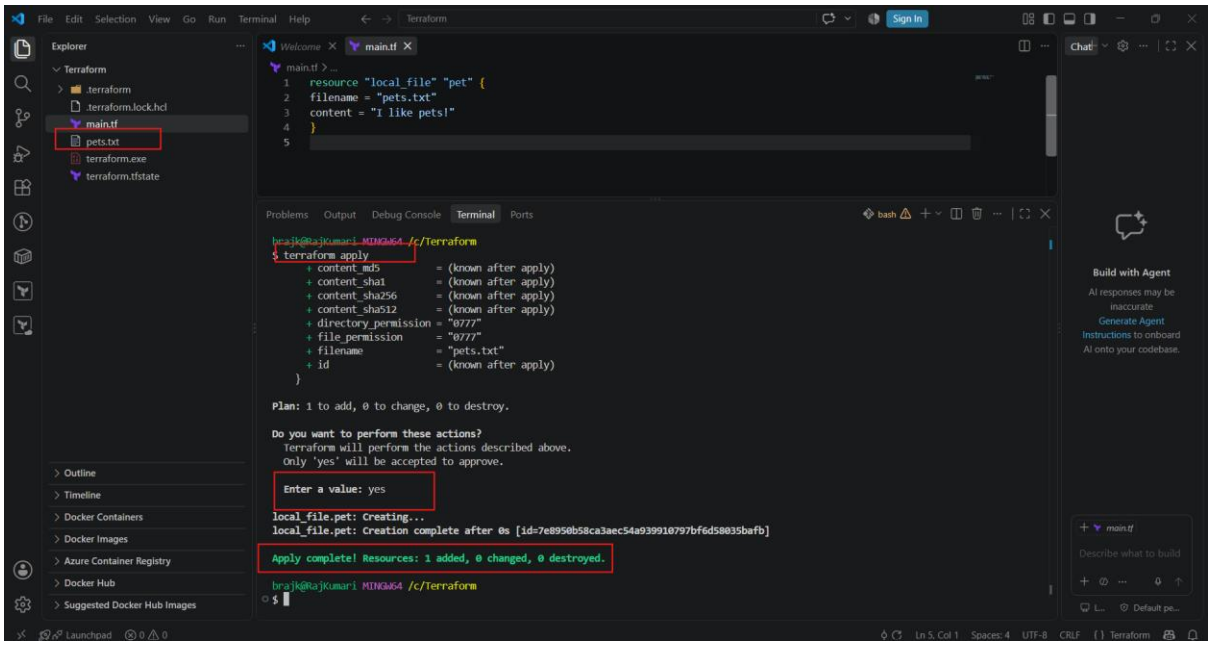
Command: $ terraform init

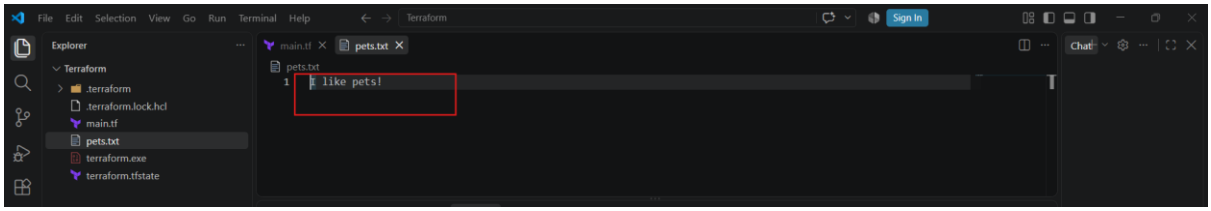


Command: $ terraform plan

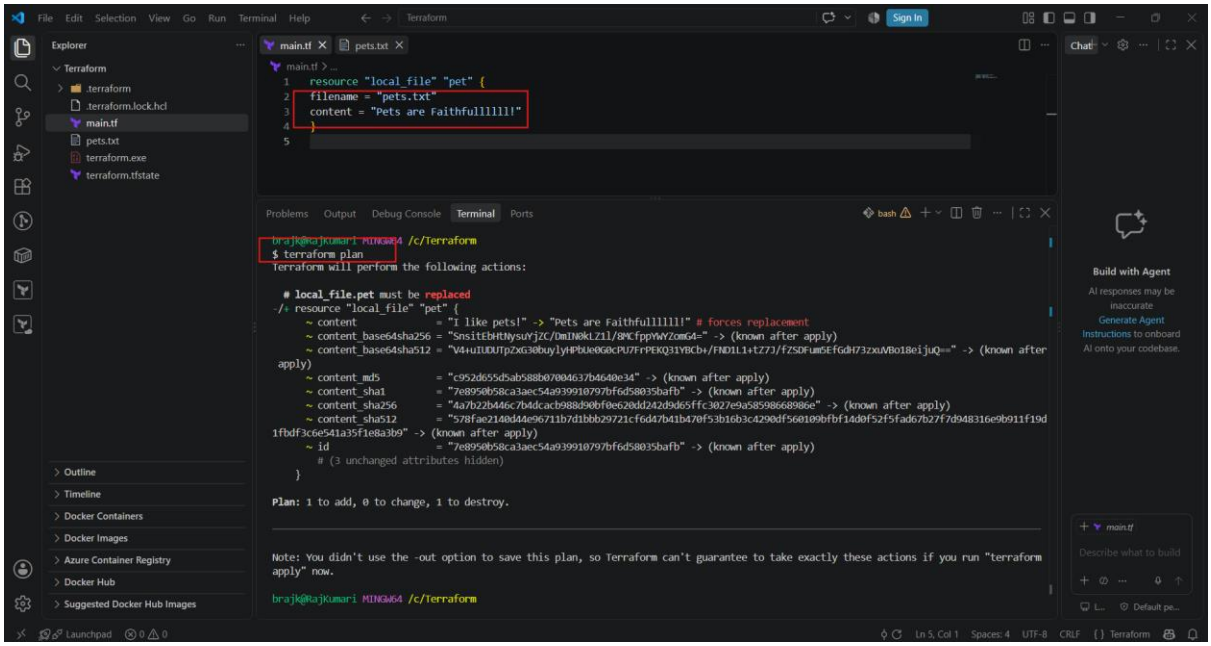


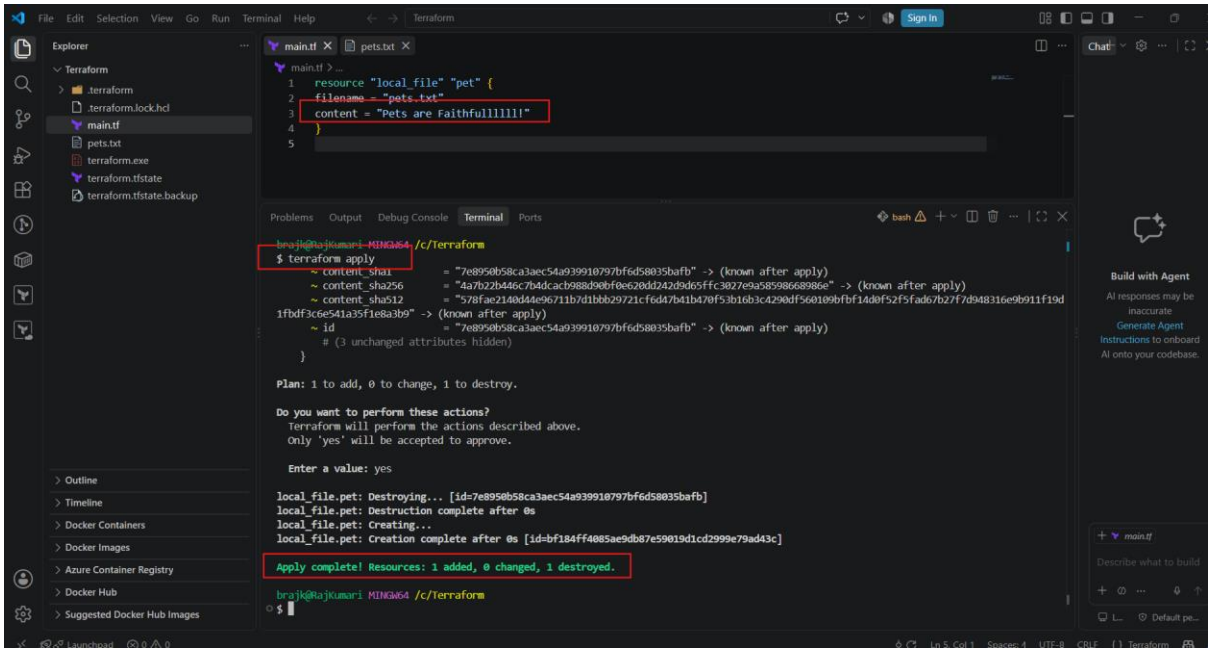
Command: $ terraform apply



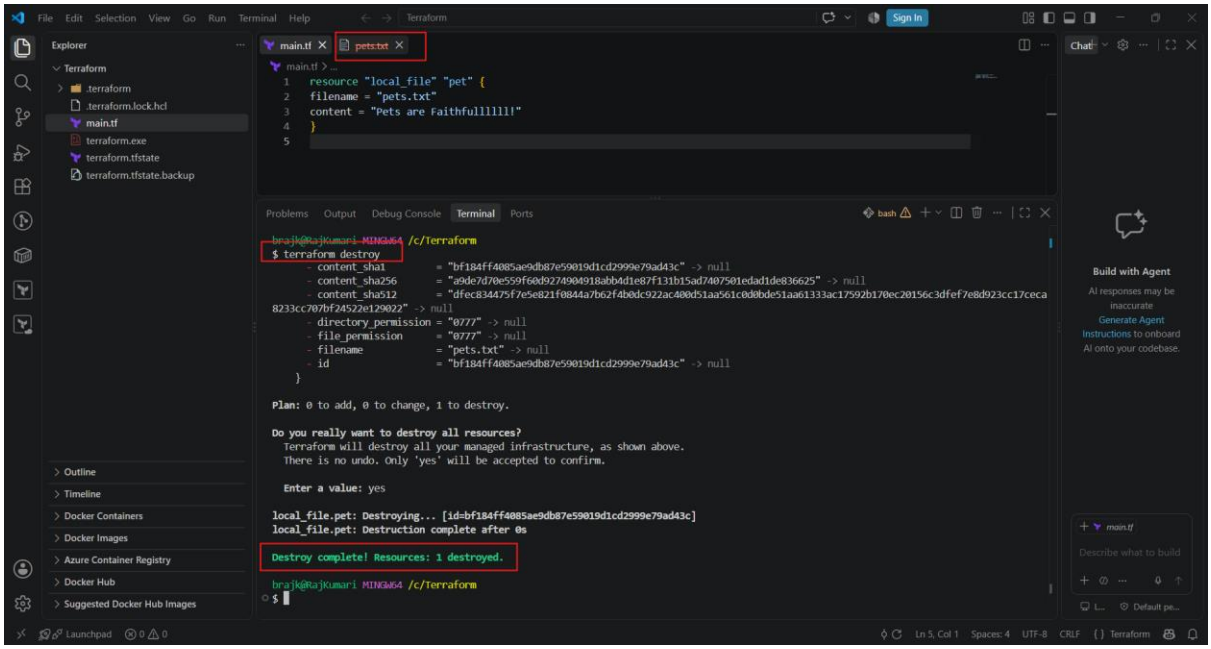


Modified main.tf file

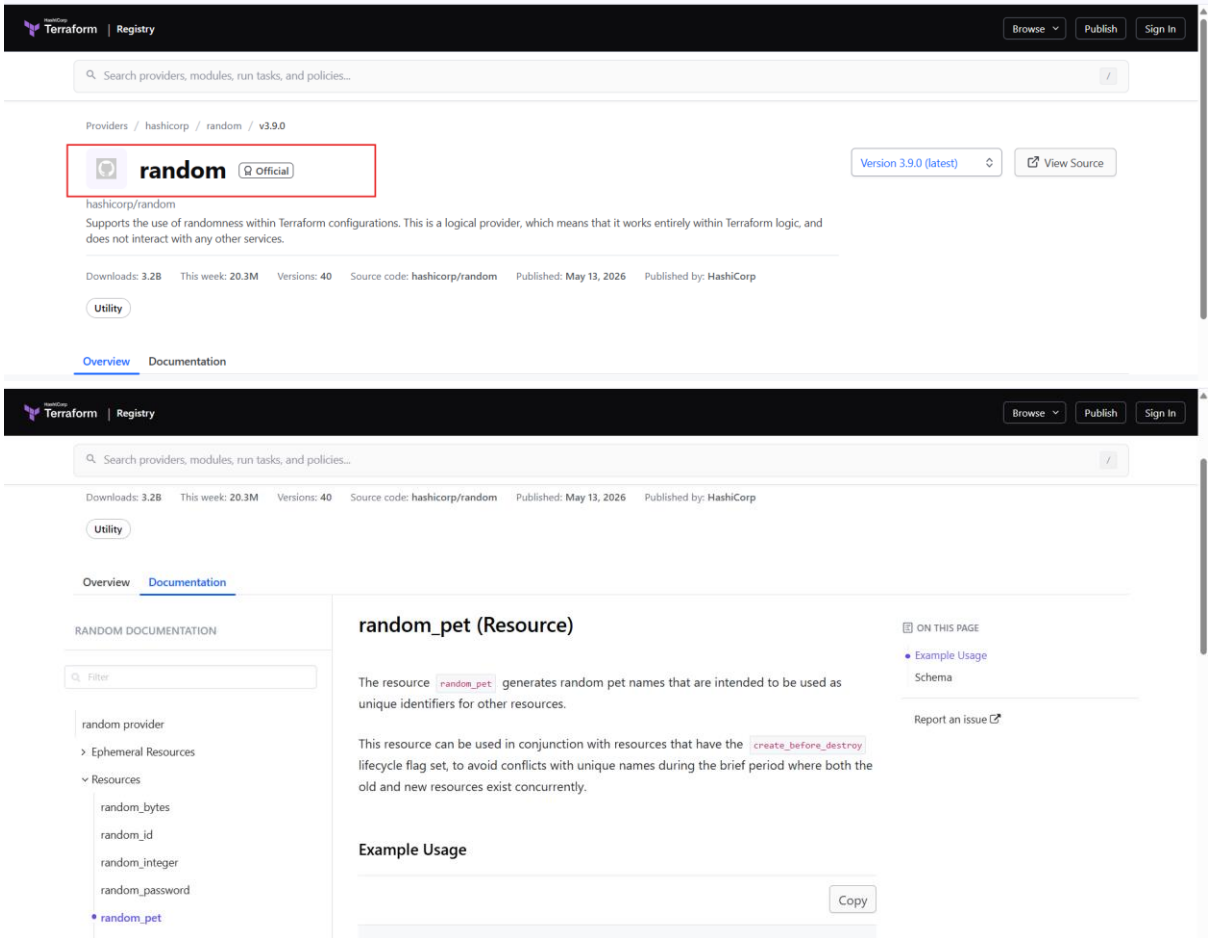




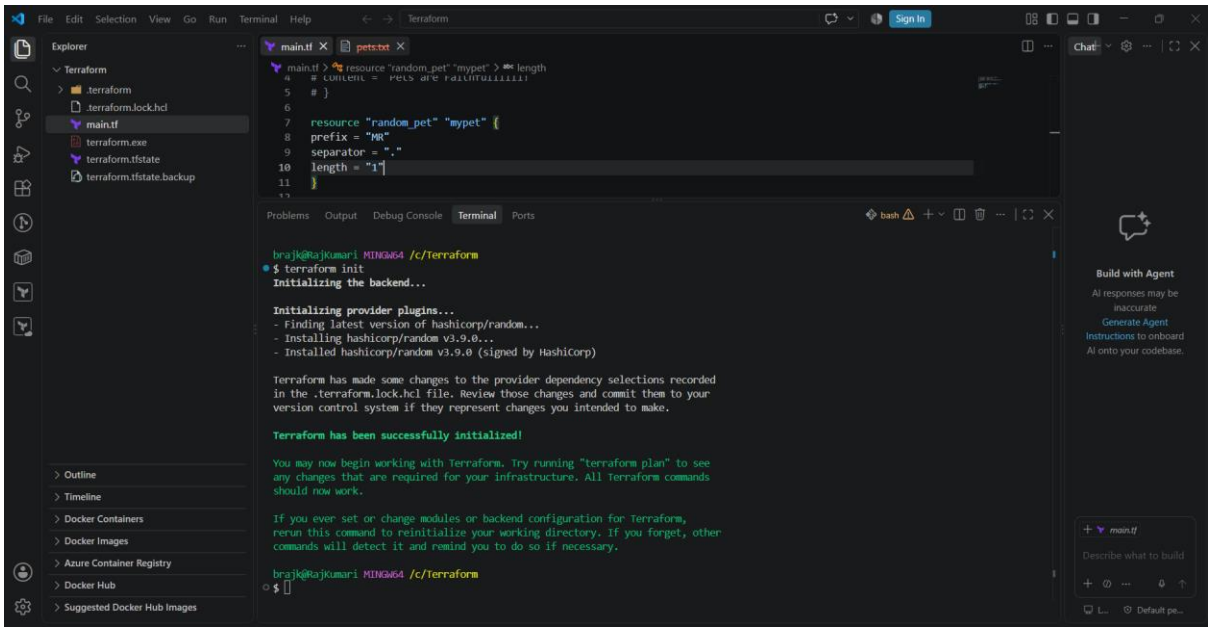
Command: $ terraform destroy



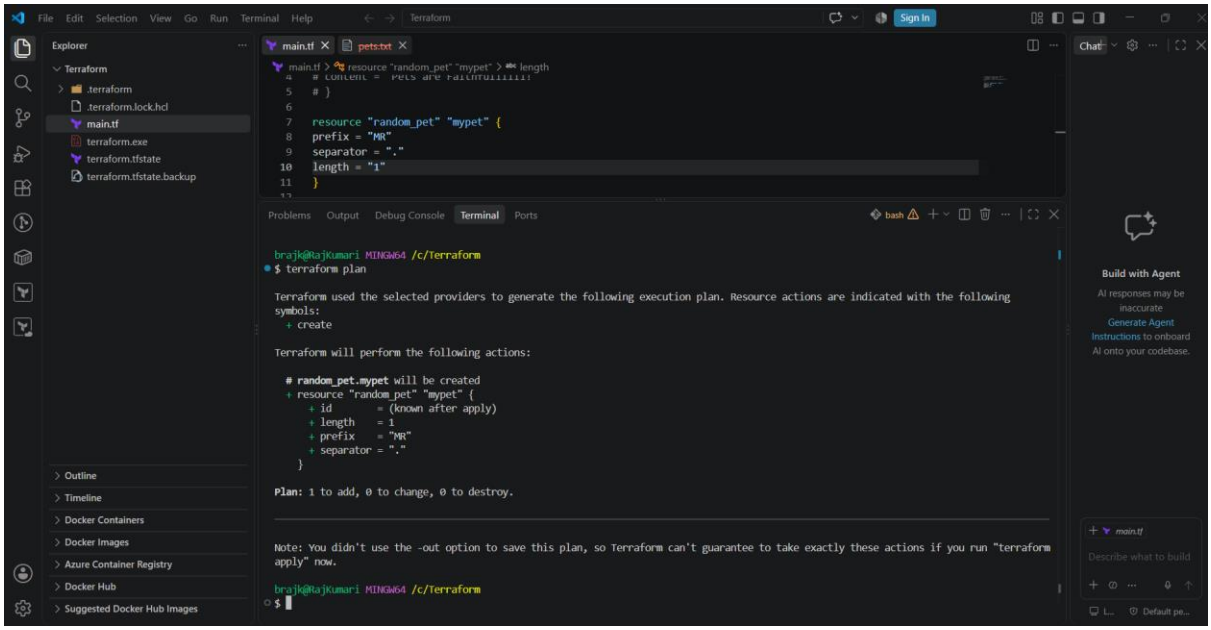
Second Template is main.tf using random provider

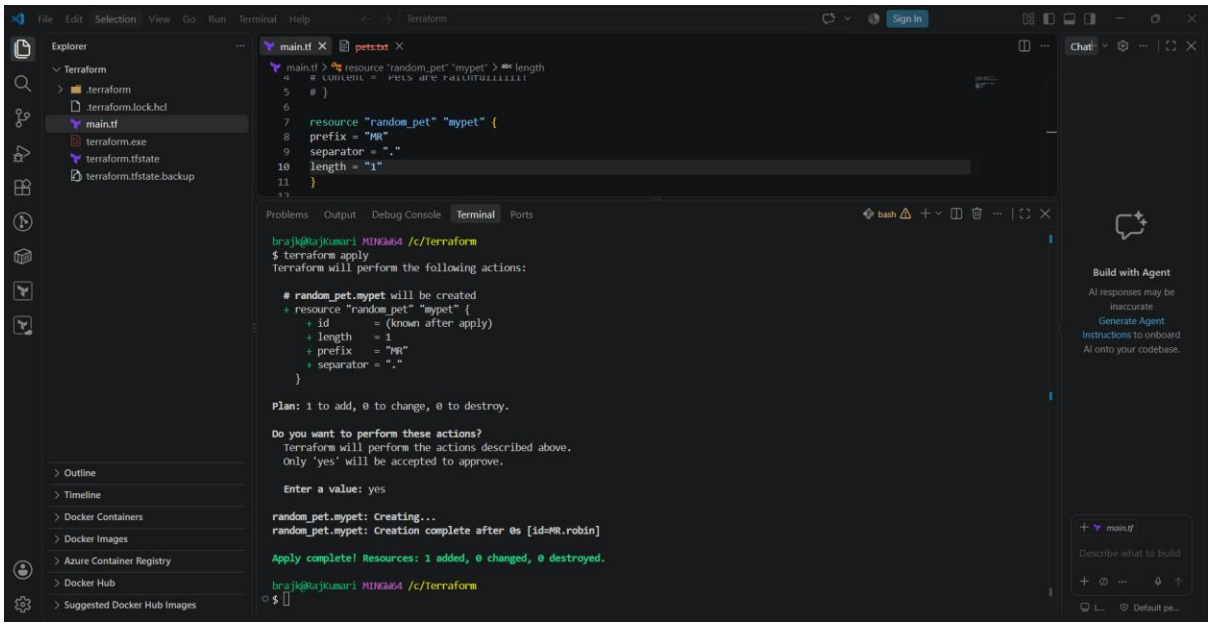


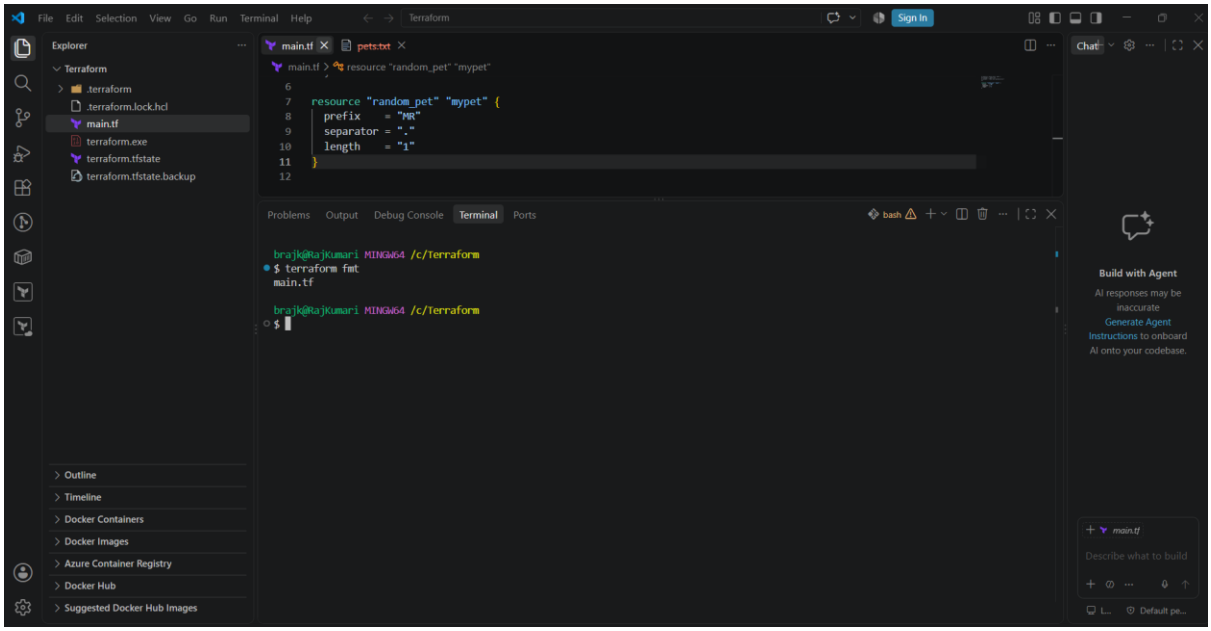
For Every provider, we need to initialise first. So, the command is “terraform init”

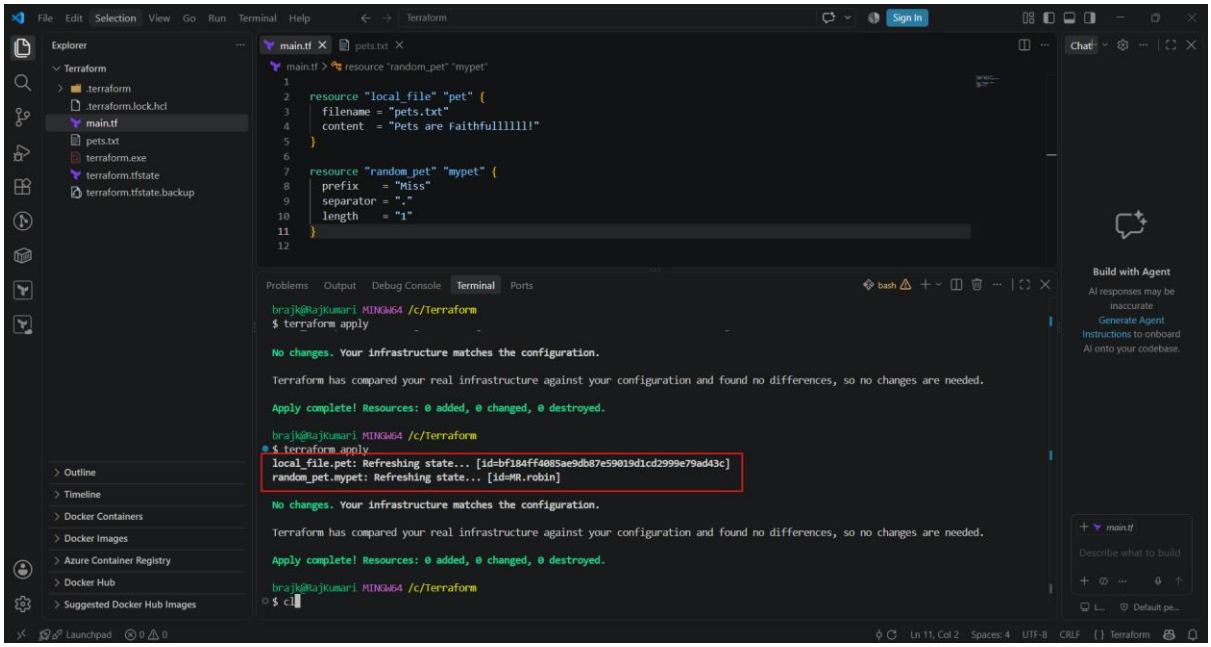


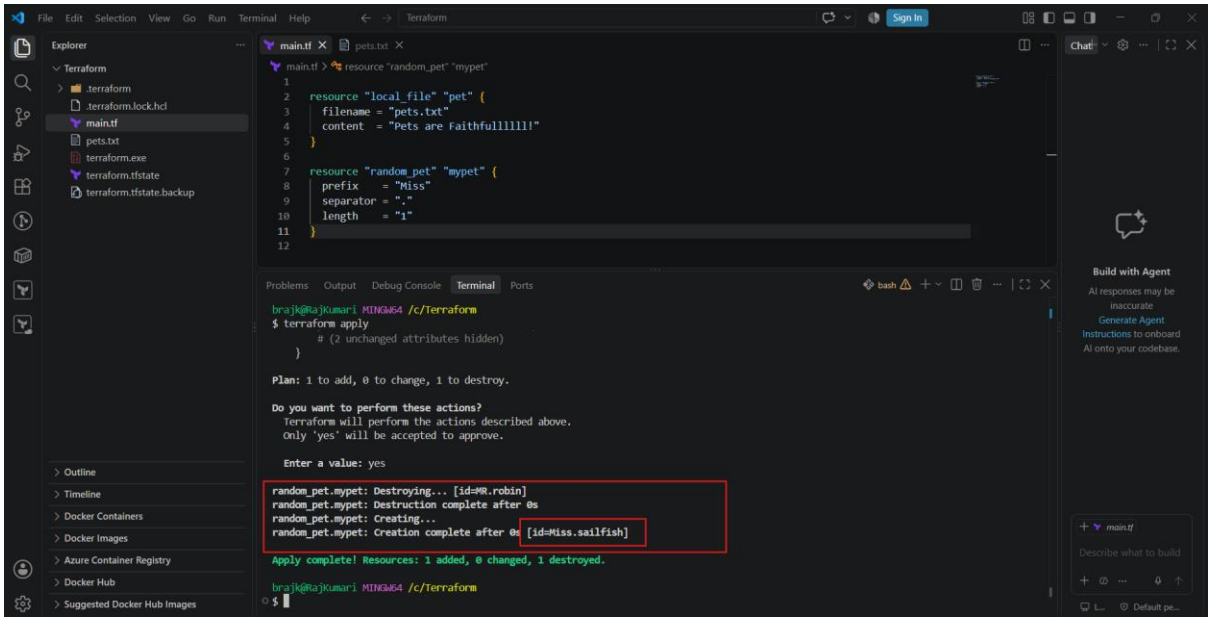
Command: $ terraformfmt $ terraform plan, $ terraform apply, $ terraform destroy


File Edit Selection View Go Run Terminal Help
Terraform
Sign In
Explorer
Terraform
.terraform
.terraform.lock.hcl
main.tf
terraform.exe
terraform.tfstate
terraform.tfstate.backup
main.tf
pets.txt
main.tf > resource "random_pet" "mypet" > length
# }
resource "random_pet" "mypet" {
prefix = "MR"
separator = "."
length = "1"
}
Problems Output Debug Console Terminal Ports
bash
brajk@RajKumari MINGW64 /c/Terraform
$ terraform plan
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
# random_pet.mypet will be created
+ resource "random_pet" "mypet" {
+ id = (known after apply)
+ length = 1
+ prefix = "MR"
+ separator = "."
}
Plan: 1 to add, 0 to change, 0 to destroy.
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
brajk@RajKumari MINGW64 /c/Terraform
$
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Describe what to build
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images


File Edit Selection View Go Run Terminal Help
Terraform
Sign In
Explorer
Terraform
.terraform
.terraform.lock.hcl
main.tf
terraform.exe
terraform.tfstate
terraform.tfstate.backup
main.tf
pets.txt
main.tf > resource "random_pet" "mypet" > length
# }
resource "random_pet" "mypet" {
prefix = "MR"
separator = "."
length = "1"
}
Problems Output Debug Console Terminal Ports
bash
brajk@RajKumari MINGW64 /c/Terraform
$ terraform apply
Terraform will perform the following actions:
# random_pet.mypet will be created
+ resource "random_pet" "mypet" {
+ id = (known after apply)
+ length = 1
+ prefix = "MR"
+ separator = "."
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
random_pet.mypet: Creating...
random_pet.mypet: Creation complete after 0s [id=MR.robin]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
brajk@RajKumari MINGW64 /c/Terraform
$
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Describe what to build
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images


File Edit Selection View Go Run Terminal Help
Terraform
Sign In
Explorer
Terraform
.terraform
.terraform.lock.hcl
main.tf
terraform.exe
terraform.tfstate
terraform.tfstate.backup
main.tf
pets.txt
main.tf > resource "random_pet" "mypet"
resource "random_pet" "mypet" {
prefix = "MR"
separator = "."
length = "1"
}
Problems Output Debug Console Terminal Ports
bash
brajk@RajKumari MINGW64 /c/Terraform
$ terraform fmt
main.tf
brajk@RajKumari MINGW64 /c/Terraform
$
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images


File Edit Selection View Go Run Terminal Help
Terraform
Sign In
Explorer
Terraform
.terraform
.terraform.lock.hcl
main.tf
pets.txt
terraform.exe
terraform.tfstate
terraform.tfstate.backup
main.tf
pets.txt
main.tf > resource "random_pet" "mypet"
resource "local_file" "pet" {
filename = "pets.txt"
content = "Pets are Faithfullllll!"
}
resource "random_pet" "mypet" {
prefix = "Miss"
separator = "."
length = "1"
}
Problems Output Debug Console Terminal Ports
bash
brajk@RajKumari MINGW64 /c/Terraform
$ terraform apply
No changes. Your infrastructure matches the configuration.
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
brajk@RajKumari MINGW64 /c/Terraform
$ terraform apply
local_file.pet: Refreshing state... [id=bf184ff4085ae9db87e59019d1cd2999e79ad43c]
random_pet.mypet: Refreshing state... [id=MR.robin]
No changes. Your infrastructure matches the configuration.
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
brajk@RajKumari MINGW64 /c/Terraform
$ cl
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images
Launchpad
Ln 11, Col 2
Spaces: 4
UTF-8
CRLF
Terraform



Terraform Command	Description
terraform init	Initializes a Terraform working directory. It downloads the required **providers**, modules, and prepares the backend for Terraform state management.
terraform plan	Creates an execution plan and shows what Terraform **will create, modify, or destroy** without actually making changes to the infrastructure.
terraform apply	Executes the Terraform plan and **creates, updates, or deletes infrastructure** according to the configuration files.
terraform provider	Used to manage and inspect Terraform **providers**. Providers allow Terraform to communicate with platforms such as AWS, Azure, GCP, Kubernetes, etc.

Example workflow

terraform init

terraform plan

terraform apply

Easy way to remember

init → Prepare
plan → Preview
apply → Execute
provider → Connect to cloud/services

For example, when using AWS:

```
provider "aws" {
  region = "us-east-1"
}
```

Here, the **AWS provider** allows Terraform to communicate with AWS and manage resources such as EC2, VPC, S3, and RDS.

Hands-on terraform Provider:

Step 1: Create a project folder

In VS Code:

File → Open Folder → New Folder

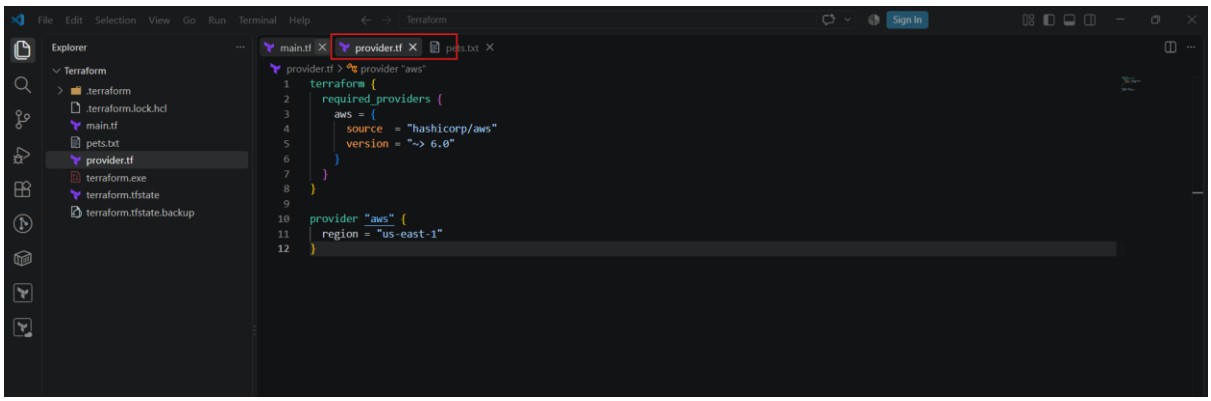
Create:

terraform-provider-demo

Step 2: Create provider.tf

In the VS Code Explorer, create:

provider.tf



Step 3: Create main.tf

Create another file:

main.tf

For a very simple provider test, we'll use the AWS caller identity data source:

data "aws_caller_identity" "current" {}

This **does not create an AWS resource**. It simply checks whether Terraform can communicate with AWS using the provider.

Step 3: Open VS Code Terminal

Click:

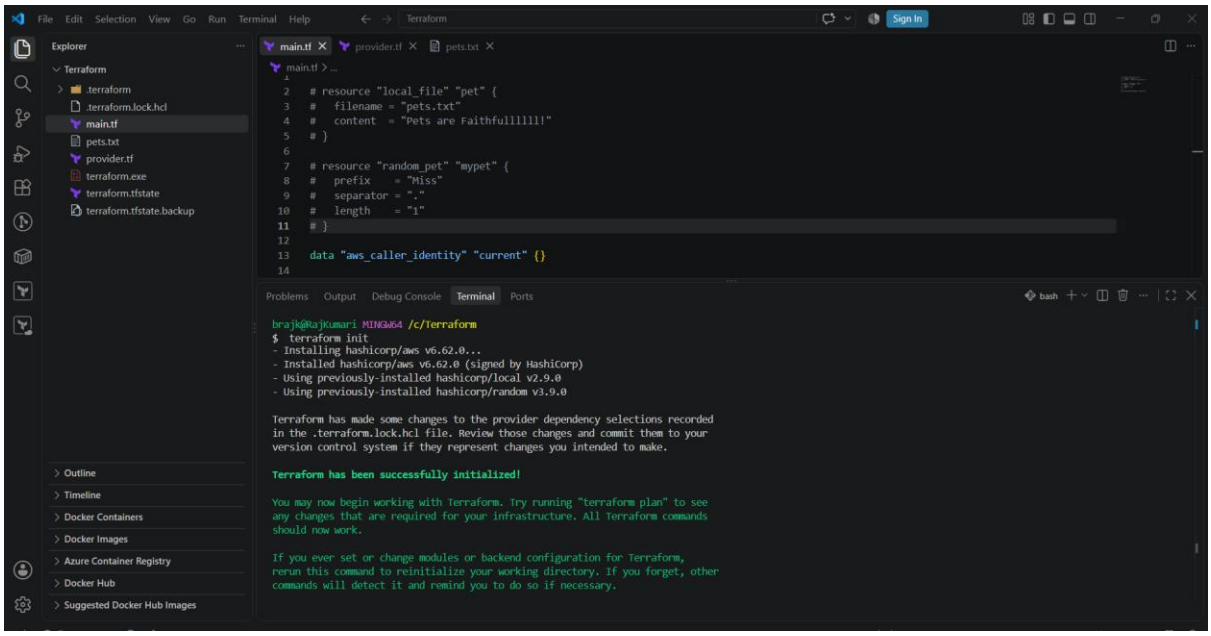
Terminal → New Terminal

You should see something like:

C:\Users\YourName\terraform

Step 4: Initialize Terraform

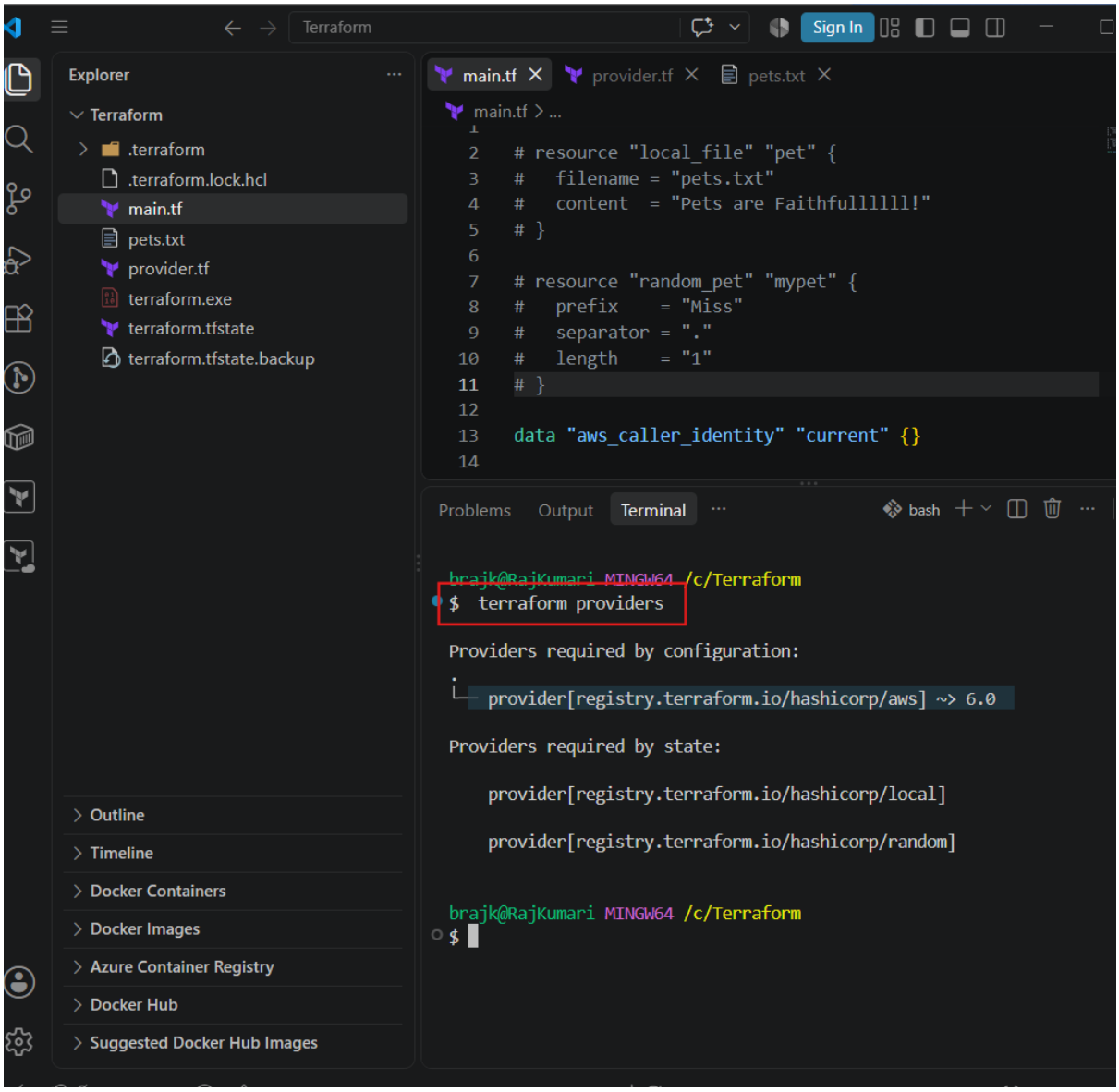
Run: $ terraform init



Step 5: Check the provider

Run:

terraform providers



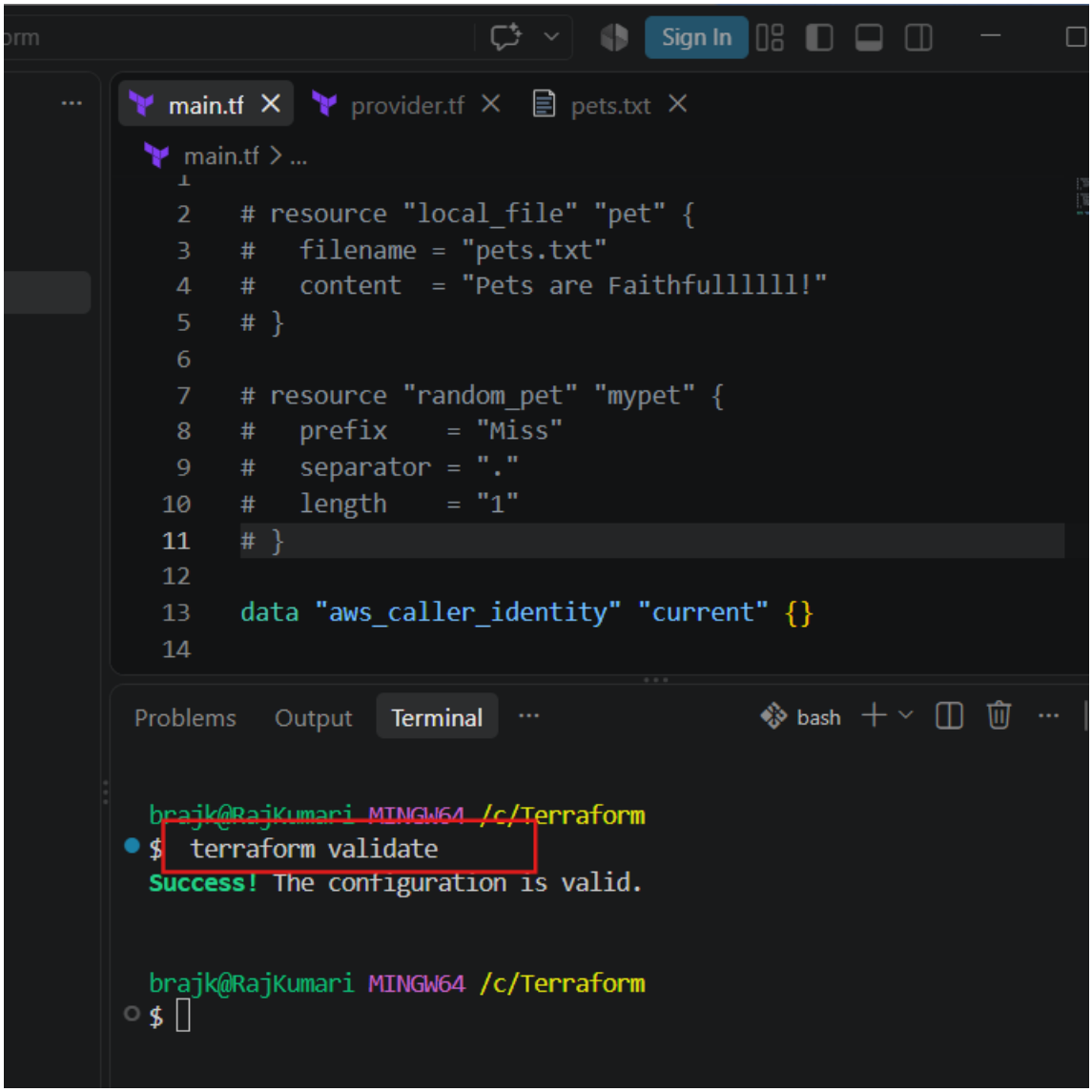
Step 7: Validate

Run:

terraform validate

Expected:

Success! The configuration is valid.



Step 8: Run Plan

Run:

terraform plan

Because we're only checking AWS identity, Terraform should not plan to create an EC2, S3 bucket, etc.

Step 9: Apply

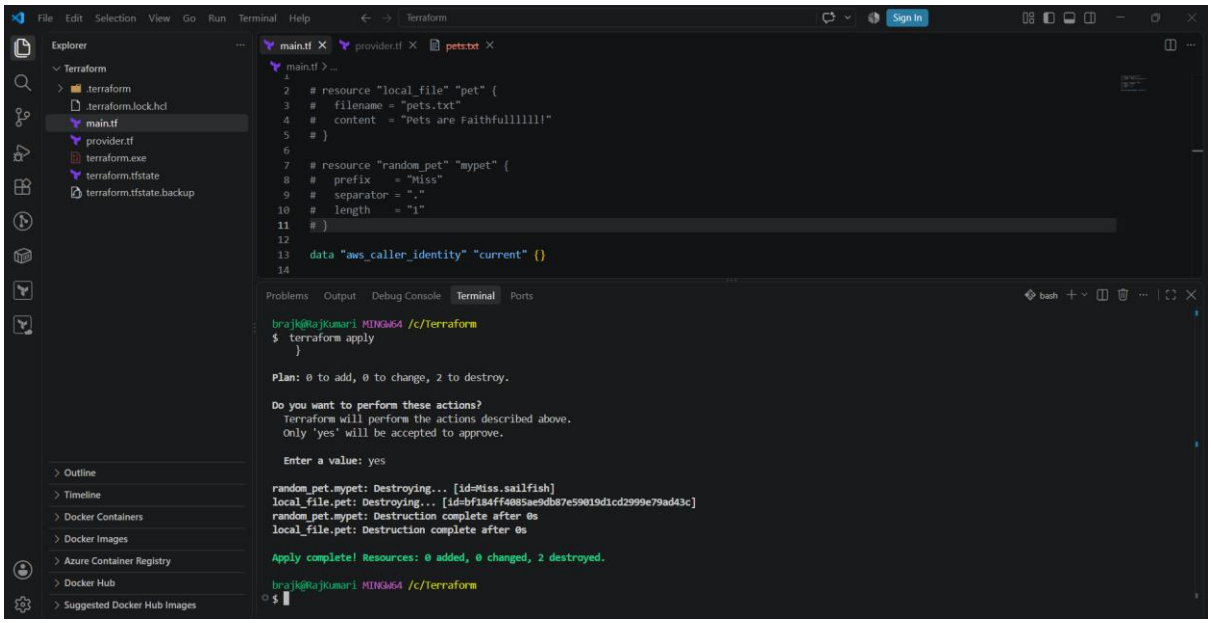
Run:

terraform apply

Type:

yes

Terraform will communicate with AWS using the AWS provider.



4. Integrate a sample Terraform template in Jenkins.

The goal is:

GitHub

↓

Jenkins

↓

Terraform Init

↓

Terraform Validate

↓

Terraform Plan

↓

Terraform Apply

↓

AWS Resource

We'll use a simple **AWS S3 bucket** example.

Step 1: Create Terraform project

In your VS Code project, create:

terraform-jenkins-demo/

```
 |
 ├── provider.tf
 ├── main.tf
 └──── Jenkinsfile
```

provider.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

main.tf

```
resource "aws_s3_bucket" "demo" {
  bucket = "terraform-jenkins-demo-123456789"
```

}

⚠ S3 bucket names must be globally unique. Change the bucket name if necessary.

Step 2: Test Terraform locally

From the VS Code terminal:

terraform init

Then:

terraform validate

Then:

terraform plan

If everything is successful, you can move to Jenkins.

Step 3: Create a GitHub repository

Create a repository such as:

terraform-jenkins-demo

Push your files:

provider.tf

main.tf

Jenkinsfile

Your GitHub repository should look like:

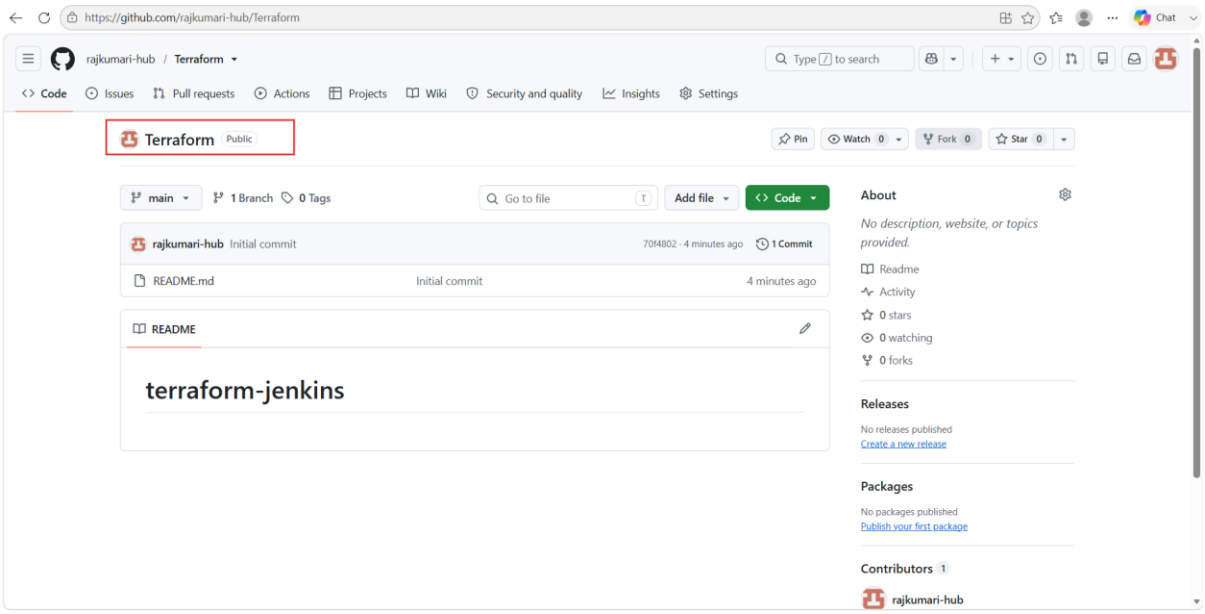
terraform-jenkins-demo

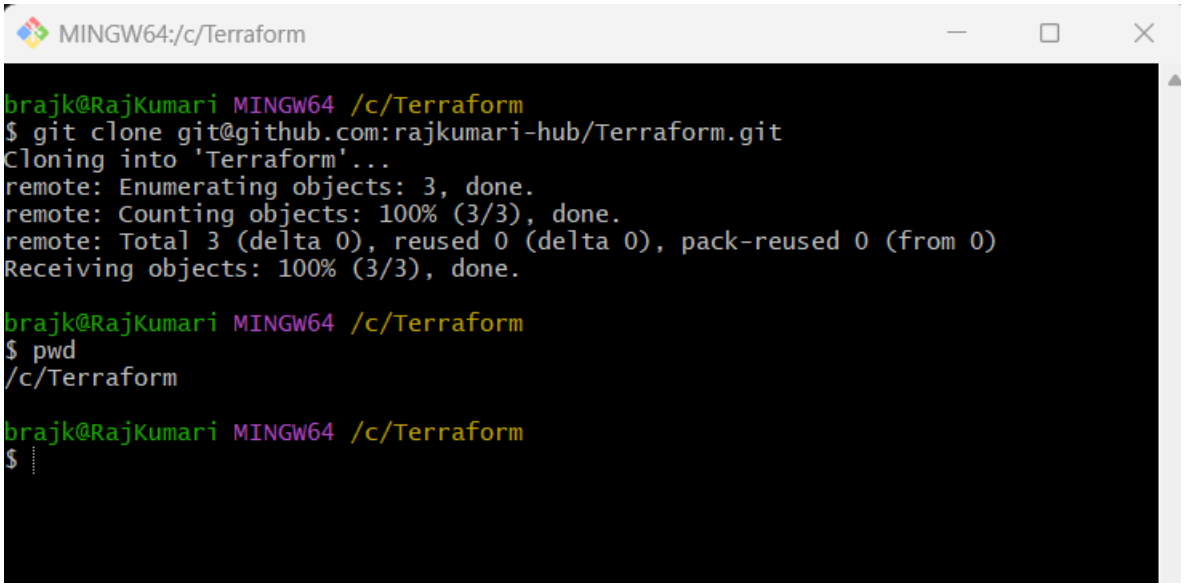
│

├── provider.tf

├── main.tf

└── Jenkinsfile

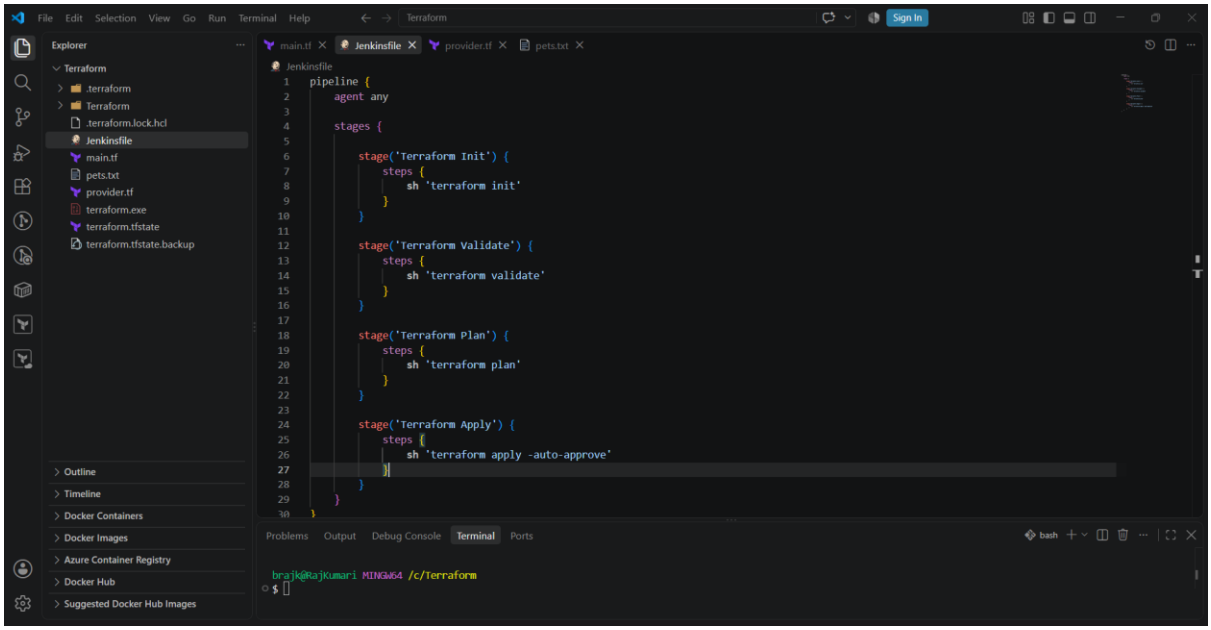




Step 4: Create Jenkinsfile

Create a file named:

Jenkinsfile



What each stage does

Stage	Purpose
Terraform Init	Downloads providers and initializes Terraform
Terraform Validate	Checks Terraform syntax/configuration
Terraform Plan	Shows what Terraform will create/change
Terraform Apply	Creates/updates the AWS infrastructure

Step 5: Install Terraform on Jenkins

Your Jenkins server must have Terraform installed.

On your Jenkins EC2/Linux server, check:

terraform version

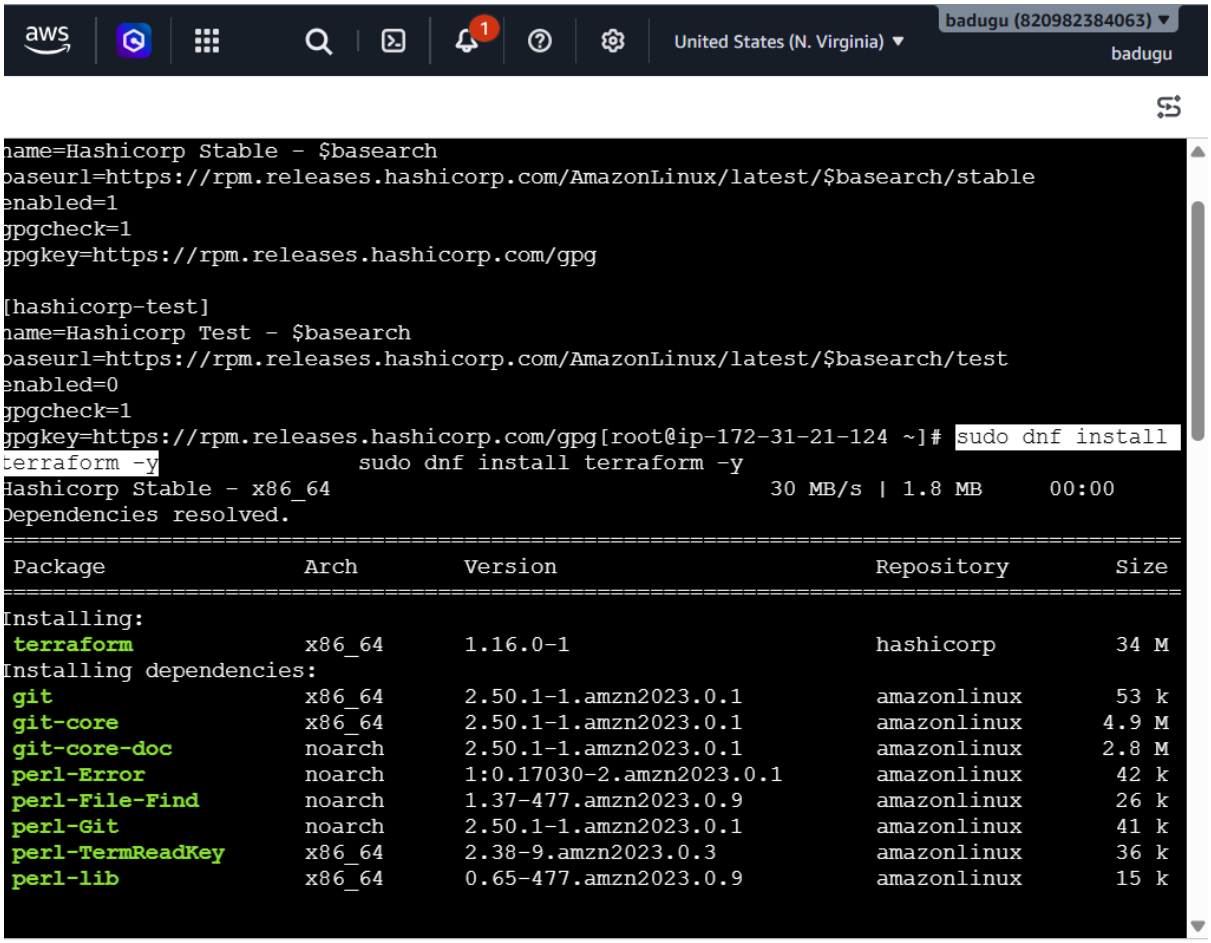
If you get:

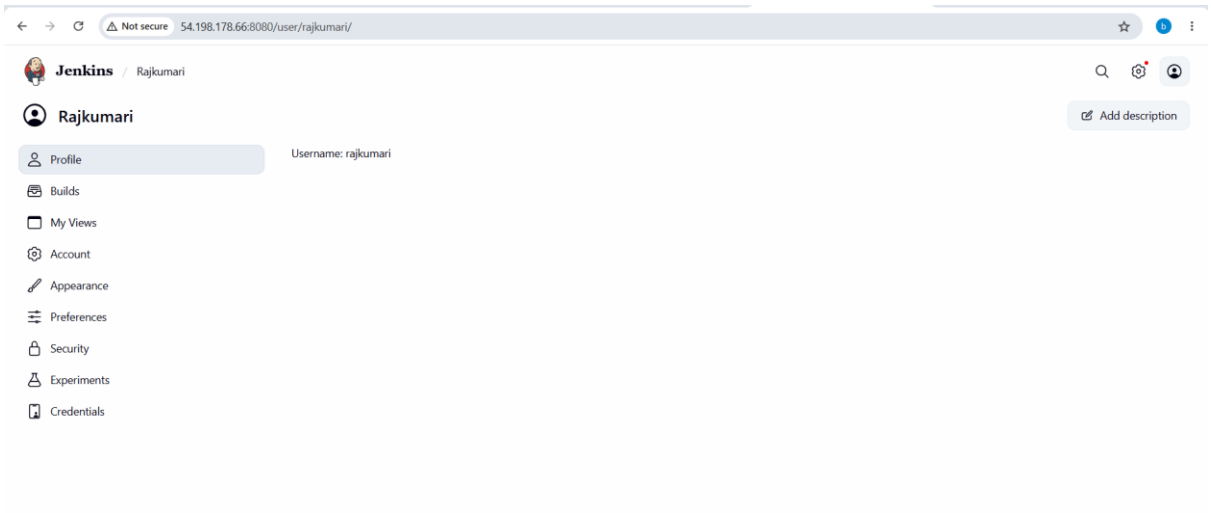
terraform: command not found

Terraform isn't installed.

After installation, verify:

terraform version





Step 6: Give Jenkins AWS permissions

This is **very important**.

Jenkins needs permission to create the S3 bucket.

If Jenkins is running on an AWS EC2 instance, the recommended approach is to attach an **IAM role** to the EC2 instance.

For this example, the role needs appropriate S3 permissions.

Then test from the Jenkins server:

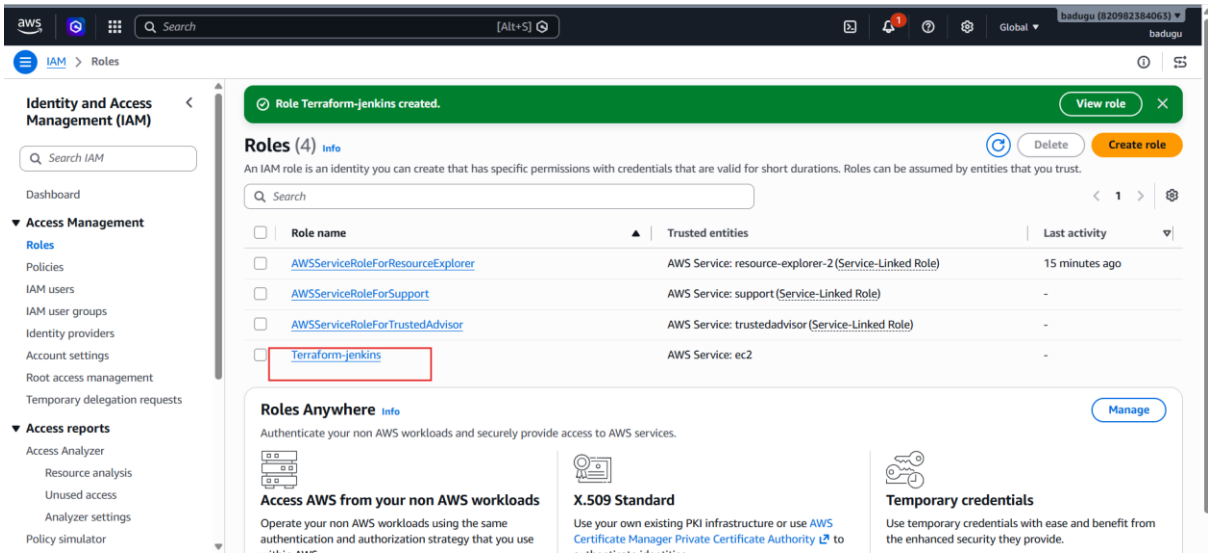
aws sts get-caller-identity

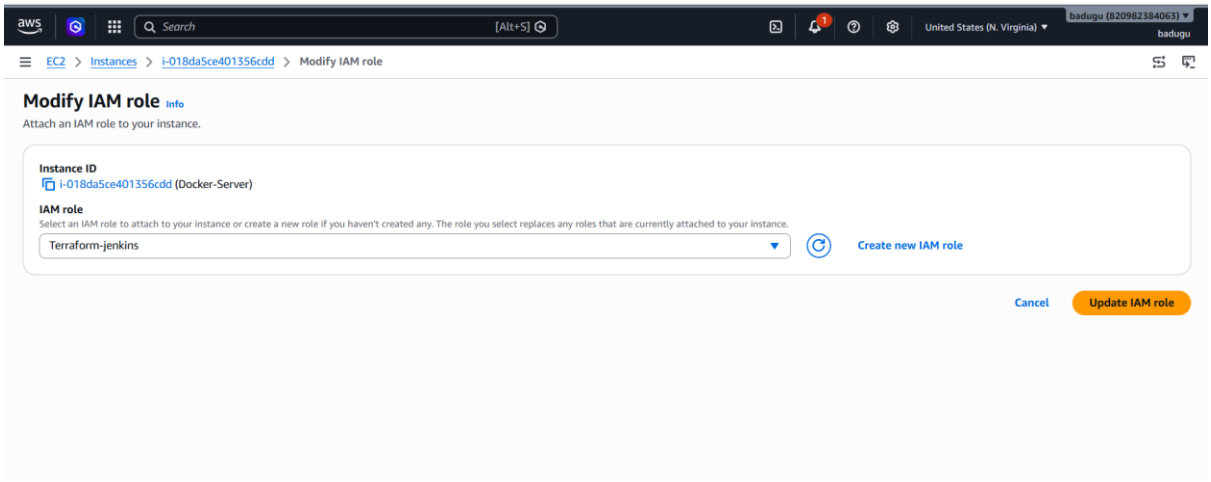
You should get something like:

Account: 123456789012

Arn: arn:aws:iam::123456789012:role/JenkinsRole

This confirms Jenkins can authenticate with AWS.






```
[root@ip-172-31-21-124 ~]# aws sts get-caller-identity
{
    "UserId": "AROA36JS5AW72VPDLSXF3:i-018da5ce401356cdd",
    "Account": "820982384063",
    "Arn": "arn:aws:sts::820982384063:assumed-role/Terraform-jenkins/i-018da5ce401356cd
"
}
[root@ip-172-31-21-124 ~]#
```

Step 7: Create Jenkins Pipeline

Open Jenkins:

Jenkins Dashboard

↓

New Item

↓

terraform-jenkins-demo

Choose:

Pipeline

Click **OK**.

Under **Pipeline**, choose:

Definition:

Pipeline script from SCM

Select:

SCM: Git

Enter your GitHub repository URL.

For example:

https://github.com/YOUR_USERNAME/terraform-jenkins-demo.git

Set:

Branch:

*/main

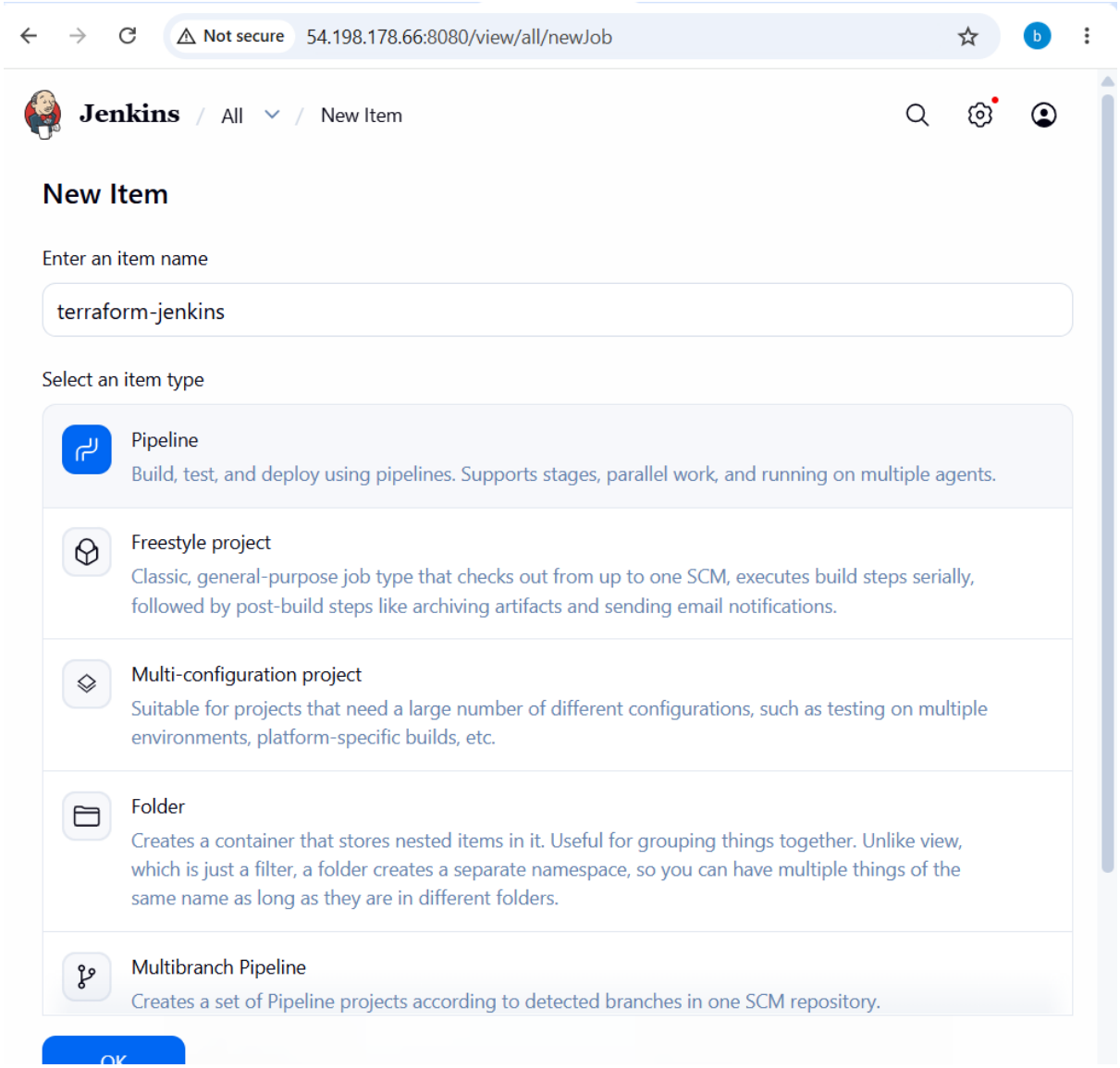
And:

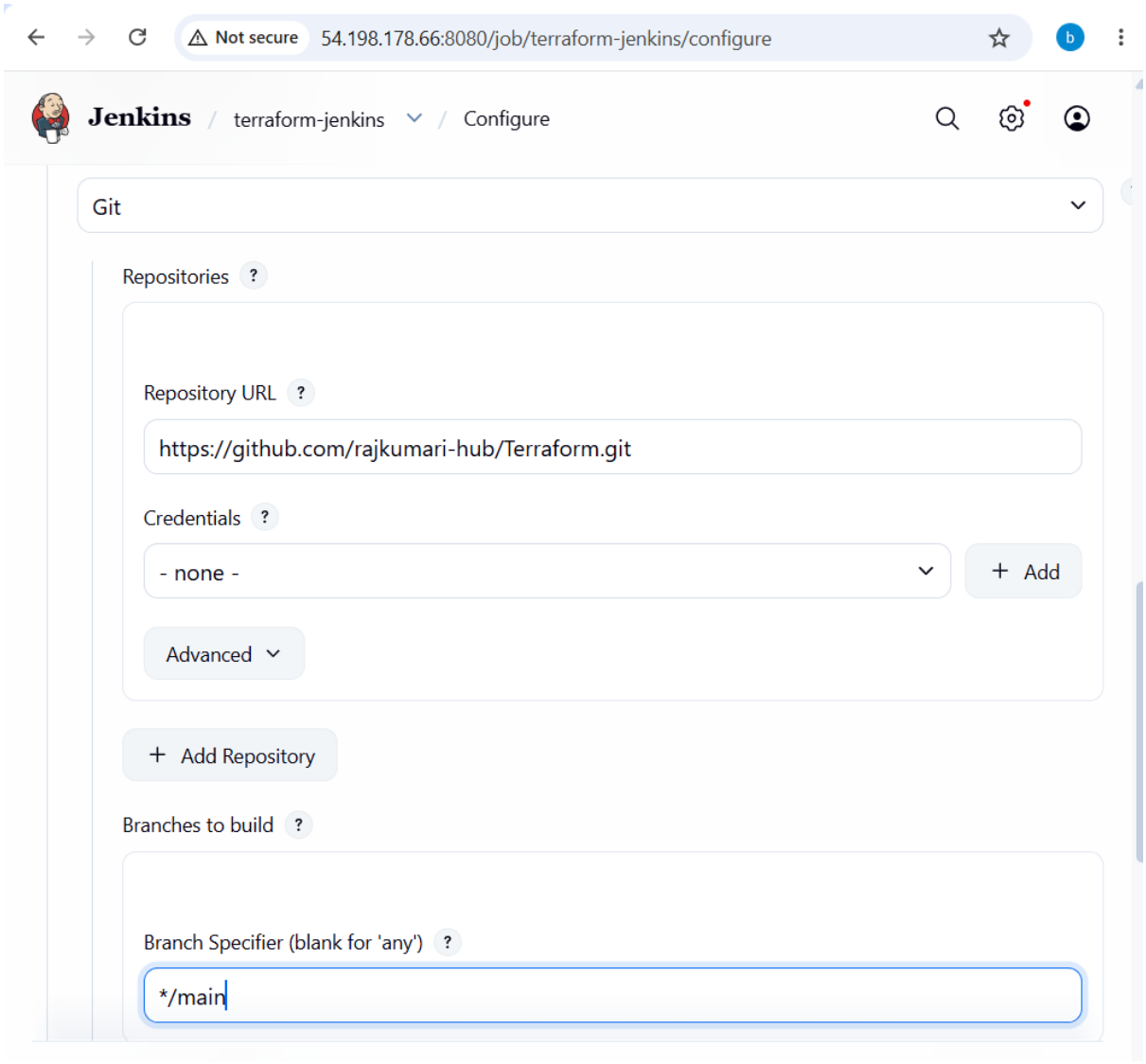
Script Path:

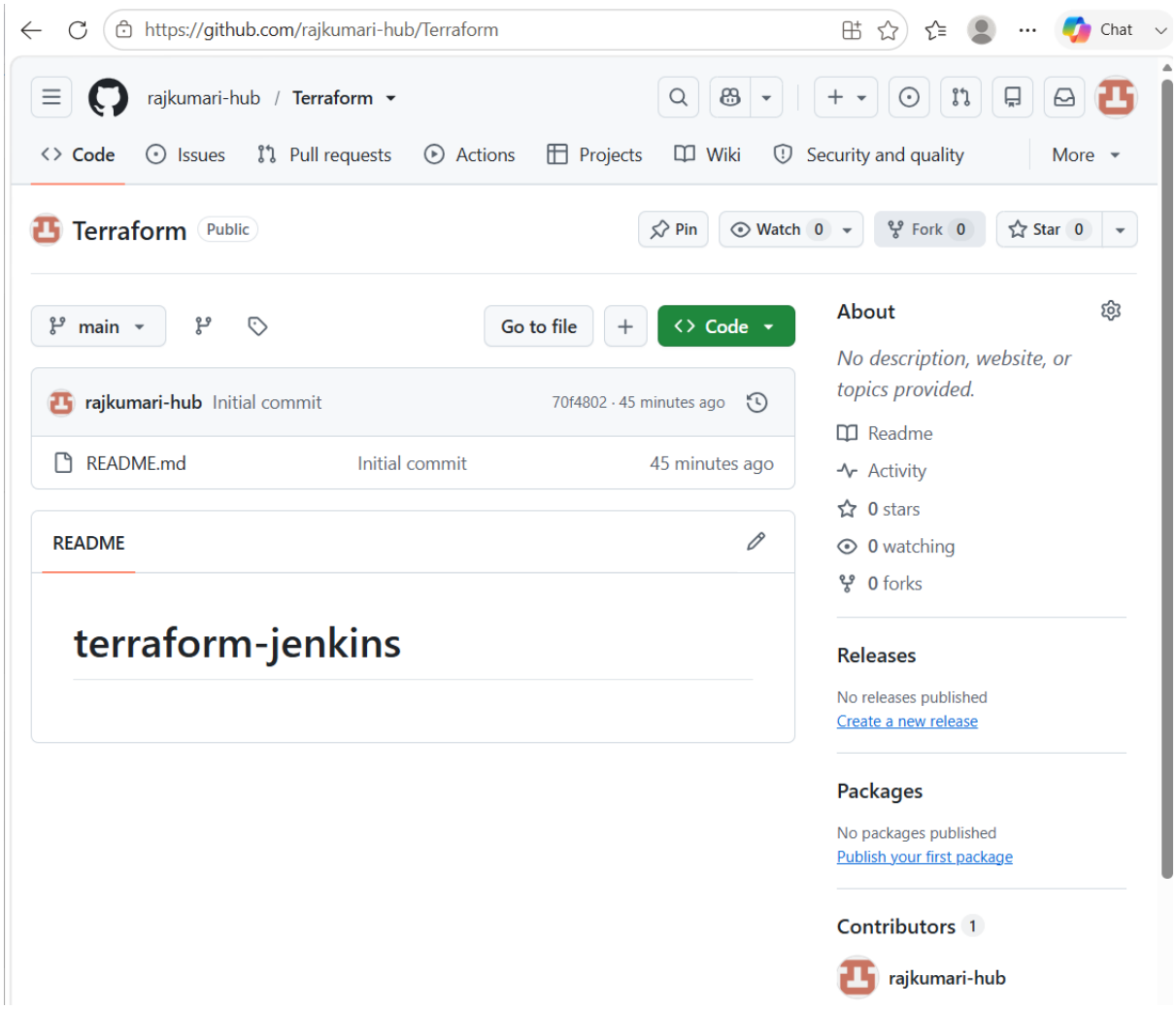
Jenkinsfile

Click:

Save




Not secure
54.198.178.66:8080/job/terraform-jenkins/configure
Jenkins
terraform-jenkins
Configure
Git
Repositories
Repository URL
https://github.com/rajkumari-hub/Terraform.git
Credentials
- none -
Add
Advanced
Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/main



Step 8: Run Jenkins Pipeline

Click:

Build Now

Jenkins will execute:

Checkout

↓

Terraform Init

↓

Terraform Validate

↓

Terraform Plan

↓

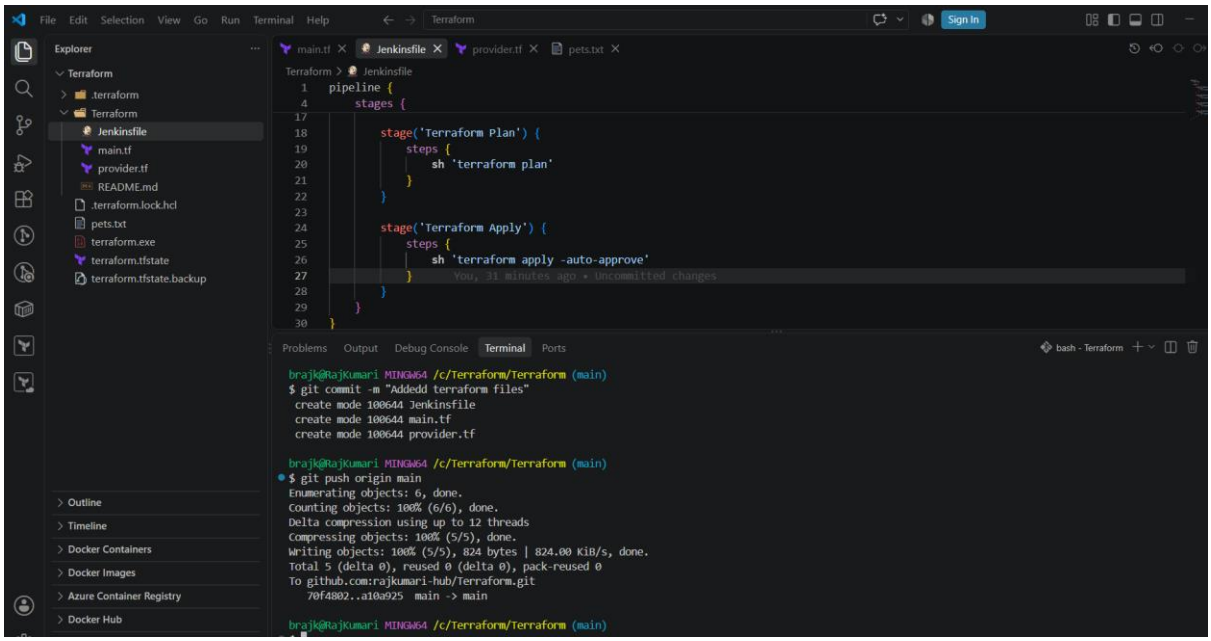
Terraform Apply

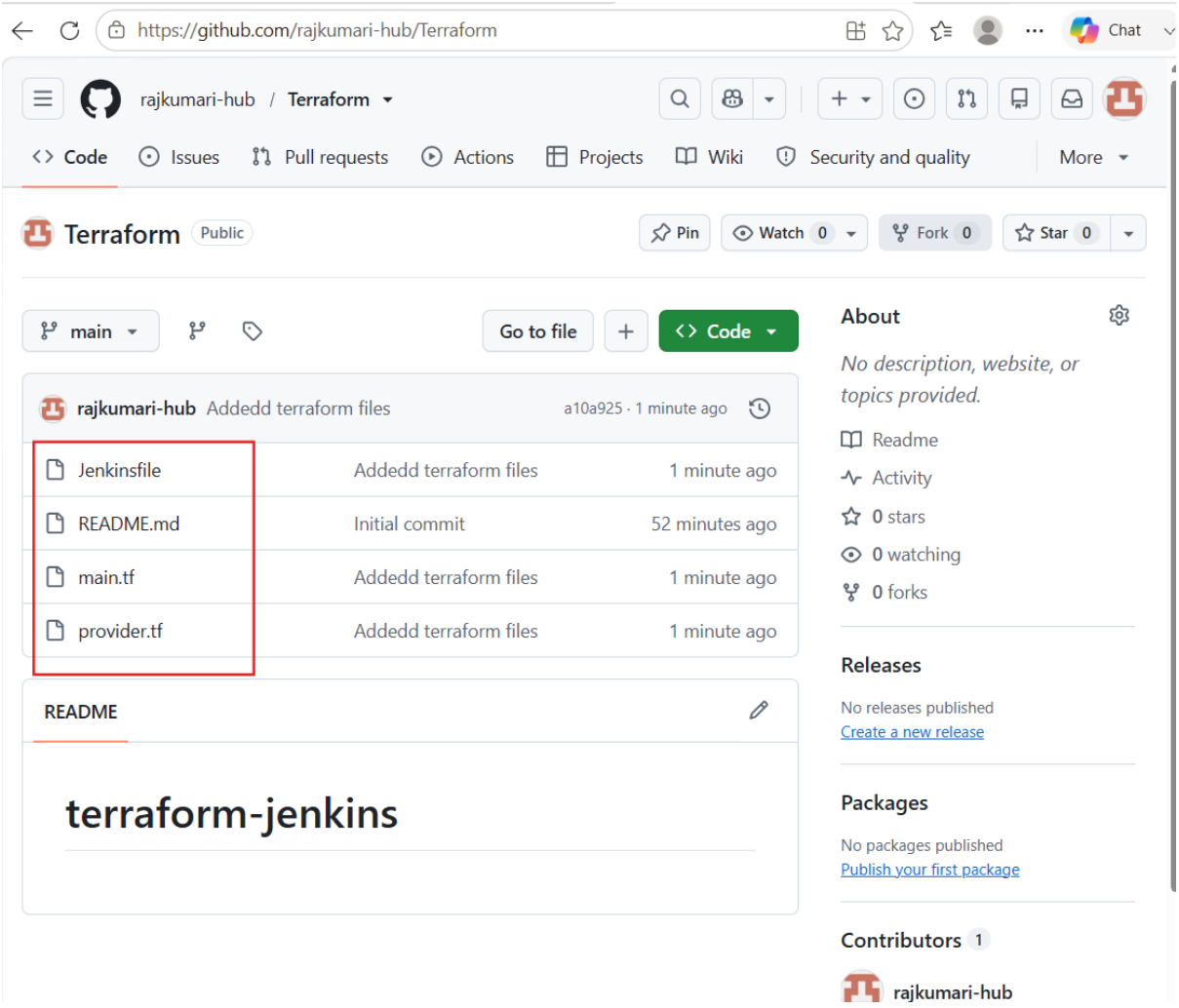
You can click:

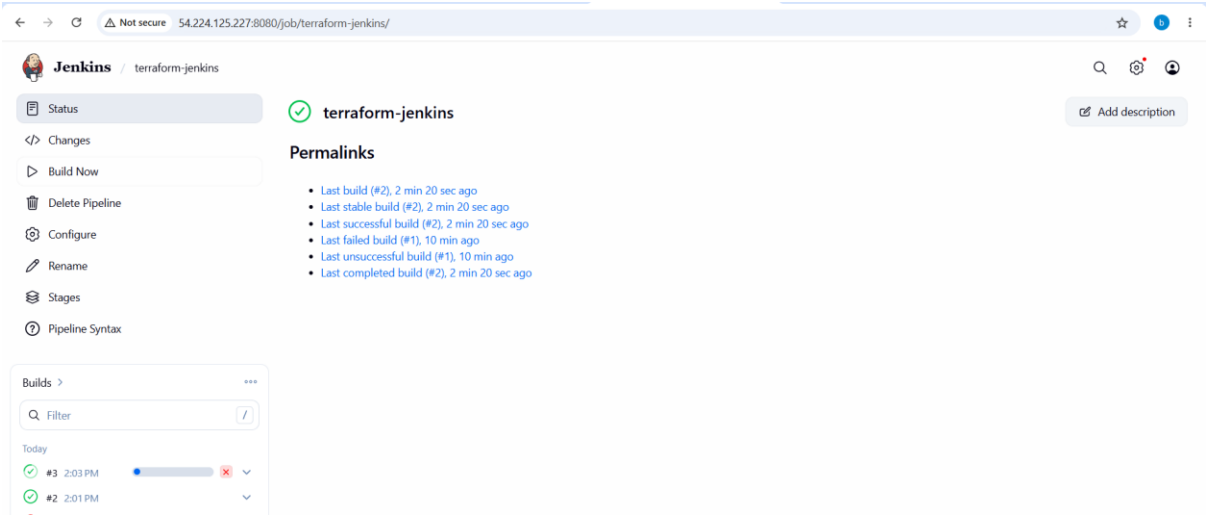
Build #1

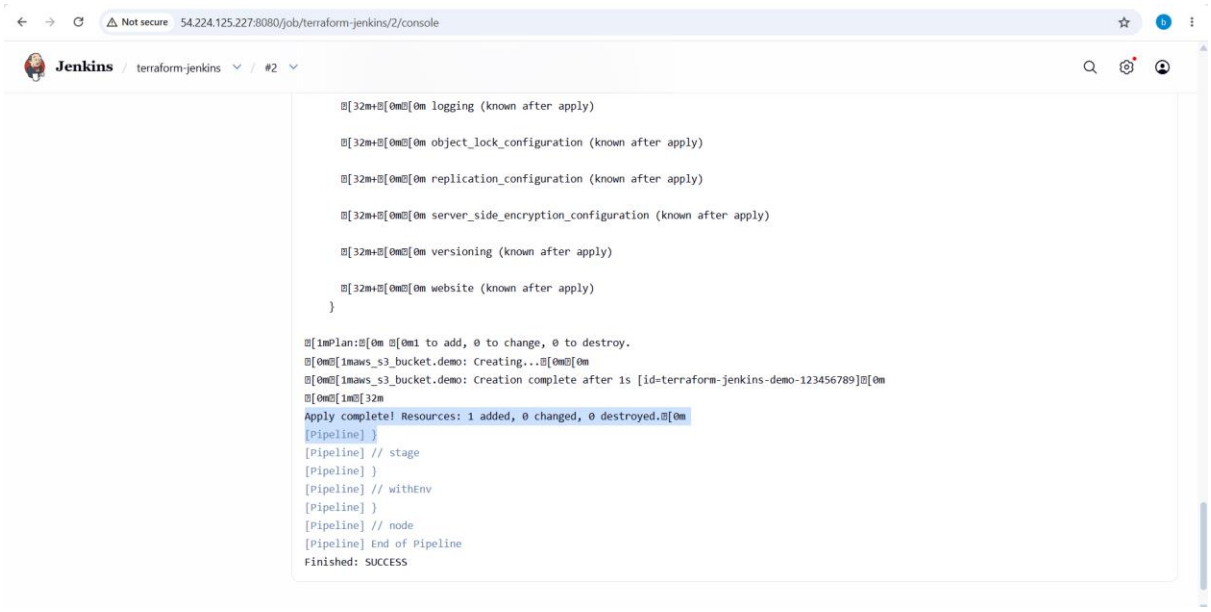
↓

Console Output



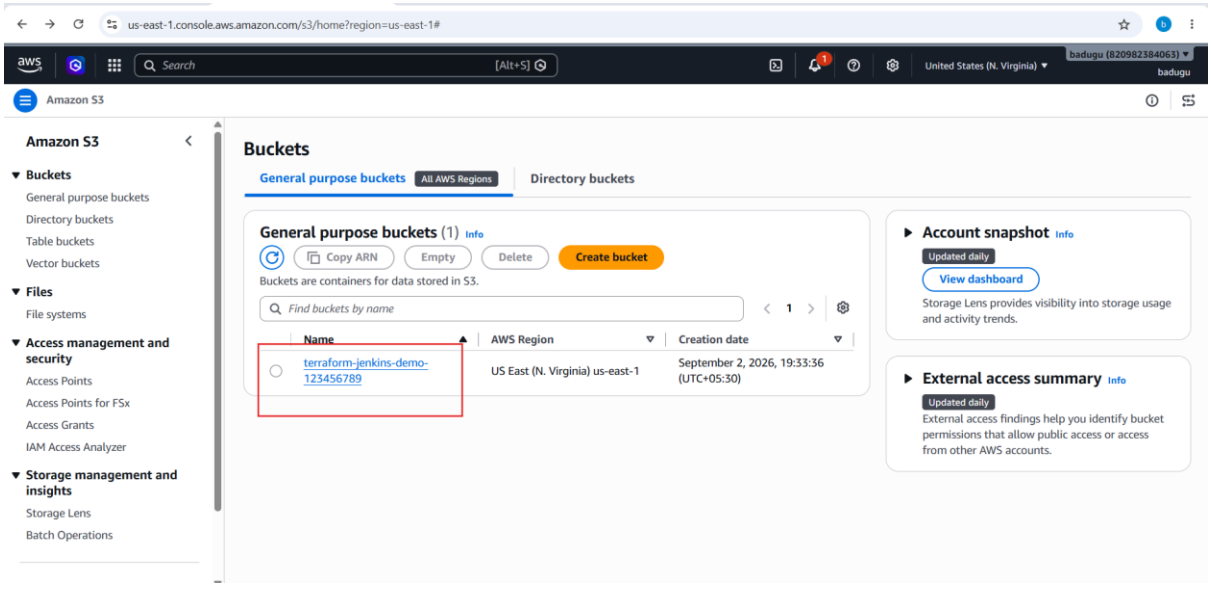

https://github.com/rajkumari-hub/Terraform
Chat
rajkumari-hub / Terraform
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
More
Terraform
Public
Pin
Watch 0
Fork 0
Star 0
main
Go to file
Code
rajkumari-hub Addedd terraform files
a10a925 · 1 minute ago
Jenkinsfile
Addedd terraform files
1 minute ago
README.md
Initial commit
52 minutes ago
main.tf
Addedd terraform files
1 minute ago
provider.tf
Addedd terraform files
1 minute ago
README
terraform-jenkins
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
rajkumari-hub


Not secure
54.224.125.227:8080/job/terraform-jenkins/
Jenkins
terraform-jenkins
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
Stages
Pipeline Syntax
terraform-jenkins
Add description
Permalinks
Last build (#2), 2 min 20 sec ago
Last stable build (#2), 2 min 20 sec ago
Last successful build (#2), 2 min 20 sec ago
Last failed build (#1), 10 min ago
Last unsuccessful build (#1), 10 min ago
Last completed build (#2), 2 min 20 sec ago
Builds
Filter
Today
#3 2:03 PM
#2 2:01 PM


Not secure 54.224.125.227:8080/job/terraform-jenkins/2/console
Jenkins terraform-jenkins #2
[32m+[0m[0m logging (known after apply)
[32m+[0m[0m object_lock_configuration (known after apply)
[32m+[0m[0m replication_configuration (known after apply)
[32m+[0m[0m server_side_encryption_configuration (known after apply)
[32m+[0m[0m versioning (known after apply)
[32m+[0m[0m website (known after apply)
}
[1mPlan:[0m [0m1 to add, 0 to change, 0 to destroy.
[0m[1maws_s3_bucket.demo: Creating...[0m[0m
[0m[1maws_s3_bucket.demo: Creation complete after 1s [id=terraform-jenkins-demo-123456789][0m
[0m[1m[32m
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


aws
Search
[Alt+S]
United States (N. Virginia)


[root@ip-172-31-29-243 ~]# aws s3 ls
2026-09-02 14:03:36 terraform-jenkins-demo-123456789
[root@ip-172-31-29-243 ~]#


us-east-1.console.aws.amazon.com/s3/home?region=us-east-1#
aws
Search
[Alt+S]
United States (N. Virginia)
badugu (820982384063)
badugu
Amazon S3
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and insights
Storage Lens
Batch Operations
Buckets
General purpose buckets
All AWS Regions
Directory buckets
General purpose buckets (1) Info
Copy ARN
Empty
Delete
Create bucket
Buckets are containers for data stored in S3.
Find buckets by name
Name
AWS Region
Creation date
terraform-jenkins-demo-123456789
US East (N. Virginia) us-east-1
September 2, 2026, 19:33:36 (UTC+05:30)
Account snapshot Info
Updated daily
View dashboard
Storage Lens provides visibility into storage usage and activity trends.
External access summary Info
Updated daily
External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.